원격 근무용
업무처리 확인 시스템

TEAM 4) 수하랑 출근할래?

팀장 : 김수하
부팀장: 이승민
팀원 : 강은비 / 김민준 / 정지수

CONTENTS

01 프로젝트 개요

02 팀 소개 및 역할

03 순서도 / ERD

04 프로젝트 수행 절차 및 방법

05 프로젝트 결과

06 시연영상

07 자체평가 및 소감

08 Q&A

chaper 01.

프로젝트 개요

AS-IS

재택근무가 보편화되면서
사무실 중심의 업무 방식으로는
팀원 간 소통, 출퇴근 관리,
업무 진행 현황 파악에 한계가 발생



TO-BE

출퇴근 체크부터 팀 협업까지
하나의 시스템으로
통합 관리 할 수 있는
원격근무 업무 시스템을 개발

chaper 02.

팀 소개 및 역할

02. 팀 소개 및 역할

김수하

- 웹 회원가입 기능 개발
- 원격 PC 접속 기능 개발
- 직원 업무 관리 기능 개발
- 관리자 승인 및 현황 관리 구현

이승민

- 직원 QR사원증 앱 개발
- 관리자 페이지 DB 연결
- 직원 및 관리자 게시판 구현
- Database 설계 보조 및 개발
- 직원 및 관리자 페이지 로그인 관련 JWT 토큰 인증 구현

강은비

- Database 설계
- 프론트(React)와 백(Flask) 설계 및 연결
- 얼굴 인식 기반 출퇴근 기능 개발
- 화상채팅 기능 개발
- 관리자 - 직원관리 메뉴 (CRUD 및 얼굴등록) 개발

김민준

- 관리자 웹페이지 UI 제작
- 실시간 채팅 구현
- 마이페이지 및 계정 정보 수정 기능 구현
- QR 코드 기반 출퇴근 기능 구현
- 관리자 - 전체 채팅 관련 채팅 로그 읽기 기능 구현

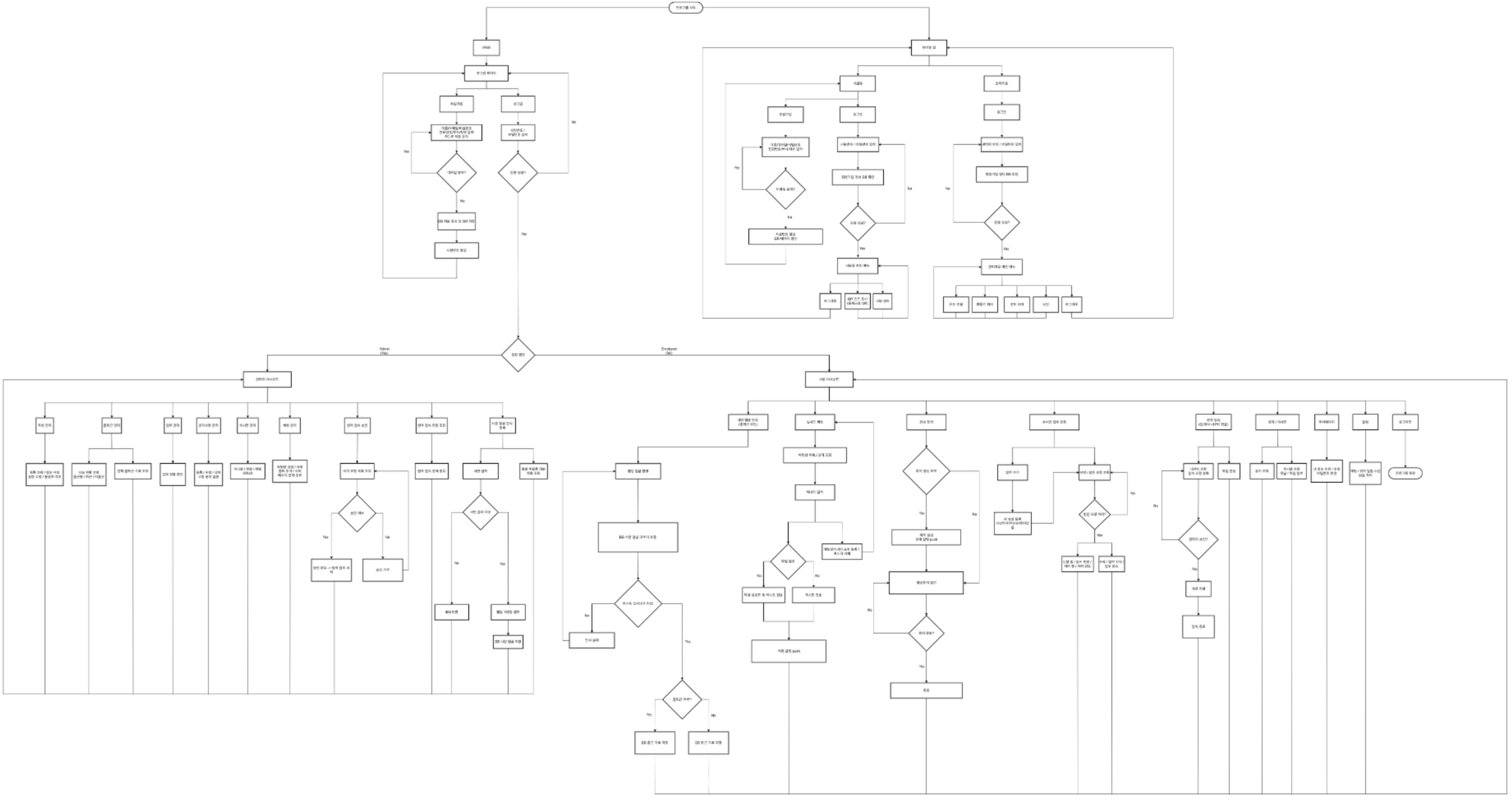
정지수

- 직원 웹페이지 UI제작
- 직원 페이지 캘린더 개발
- 관리자 / 직원 - 공지사항 기능 개발
- 관리자 - 통제실 하드웨어 및 앱 개발

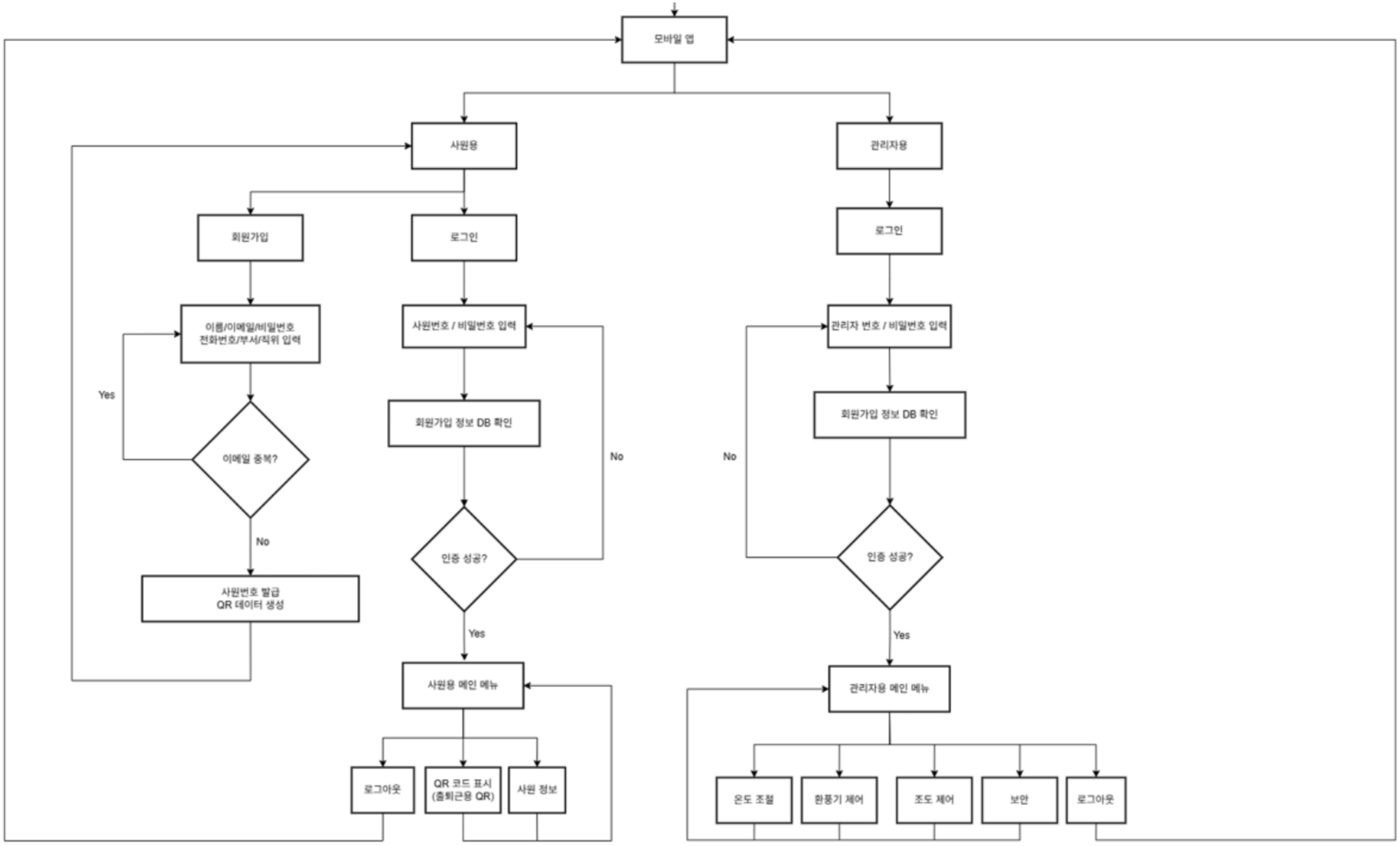
chaper 03.

순 서 도 / ERD

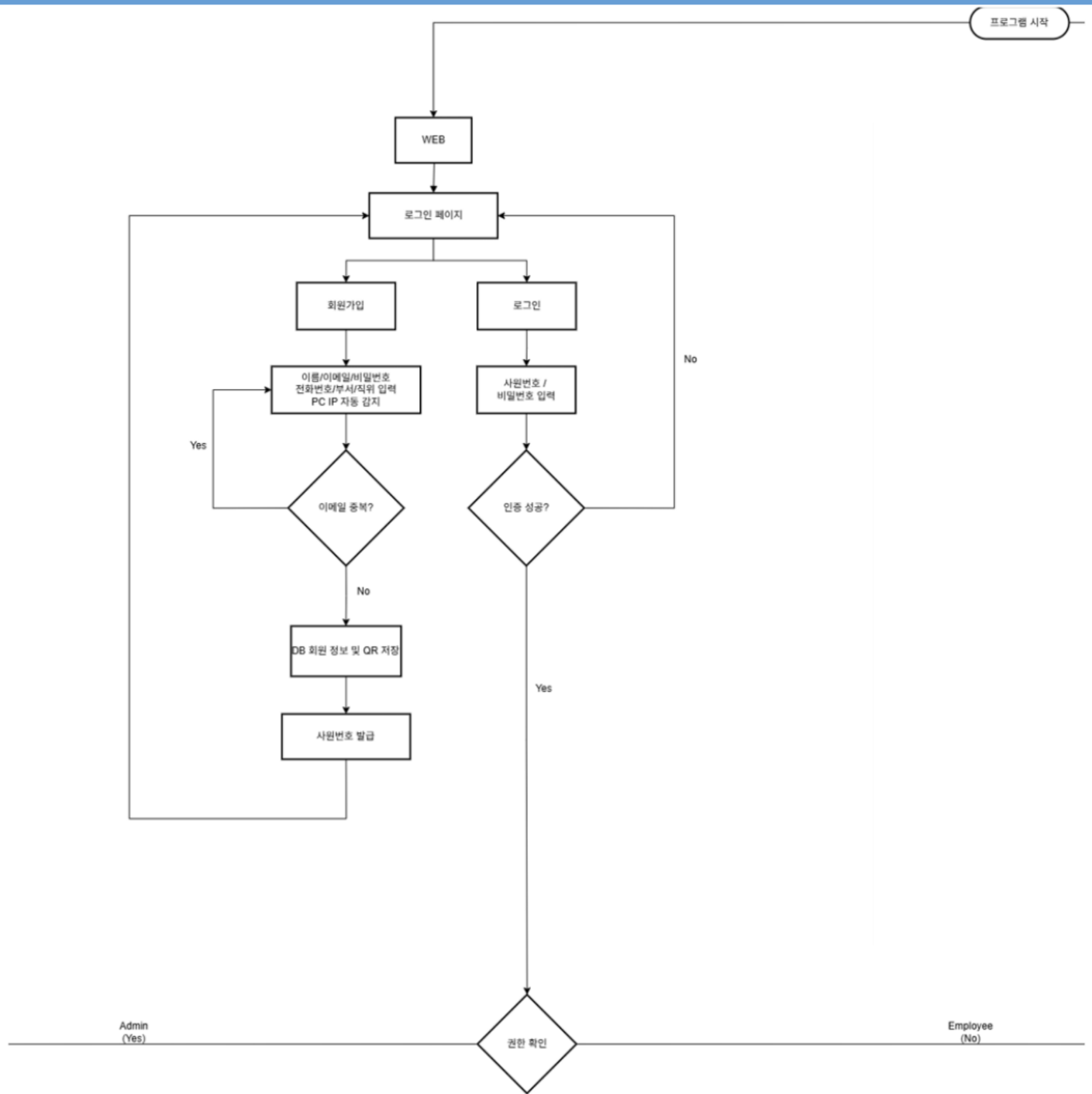
03. 순서도



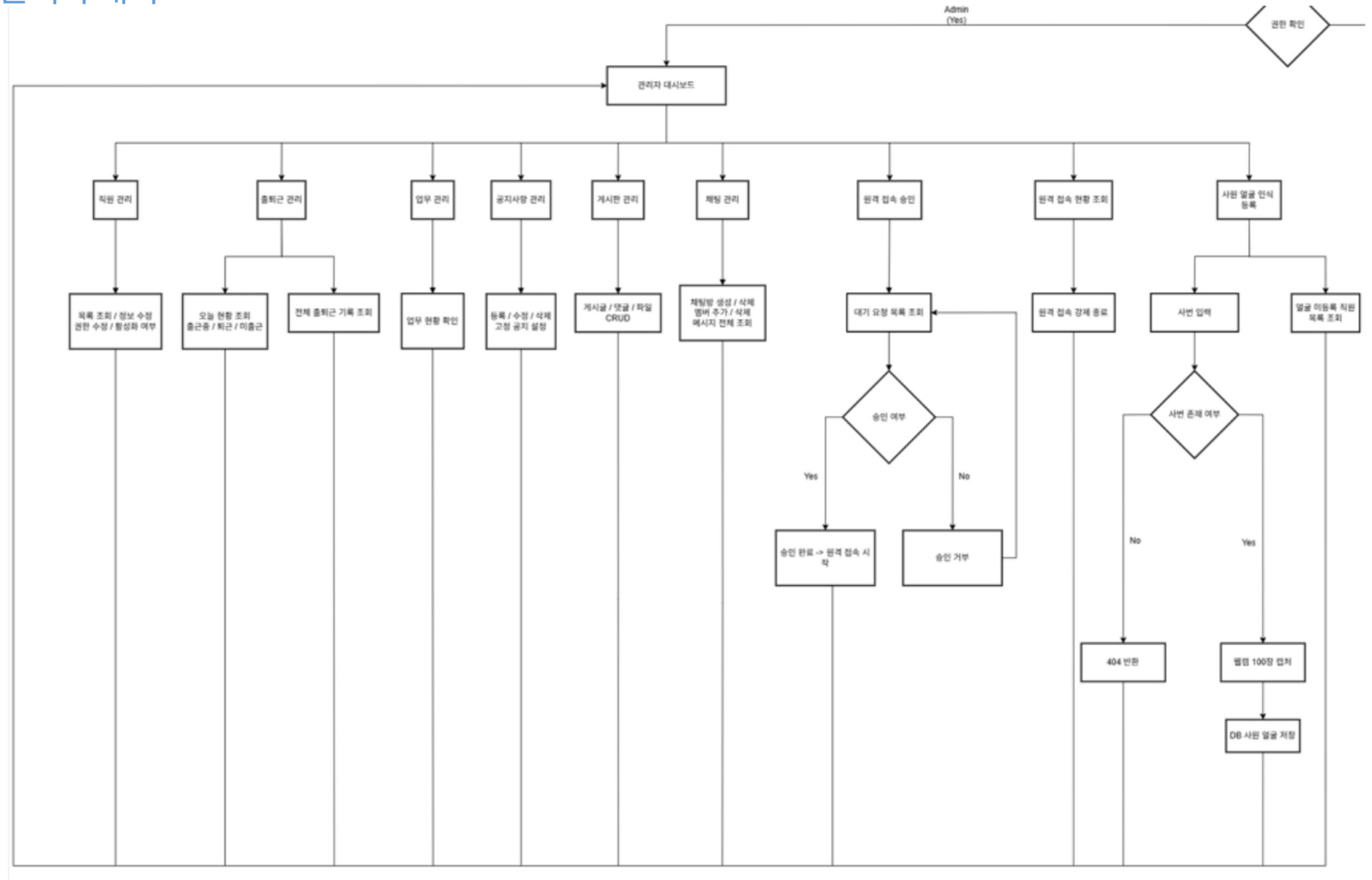
03. 순서도 - 모바일 앱


모바일 앱
사원용
관리자용
회원가입
로그인
로그인
이름/이메일/비밀번호
전화번호/부서/직위 입력
사원번호 / 비밀번호 입력
관리자 번호 / 비밀번호 입력
Yes
이메일 중복?
No
사원번호 발급
QR 데이터 생성
회원가입 정보 DB 확인
No
인증 성공?
Yes
사원용 메인 메뉴
로그아웃
QR 코드 표시
(출퇴근용 QR)
사원 정보
회원가입 정보 DB 확인
No
인증 성공?
Yes
관리자용 메인 메뉴
온도 조절
환풍기 제어
조도 제어
보안
로그아웃

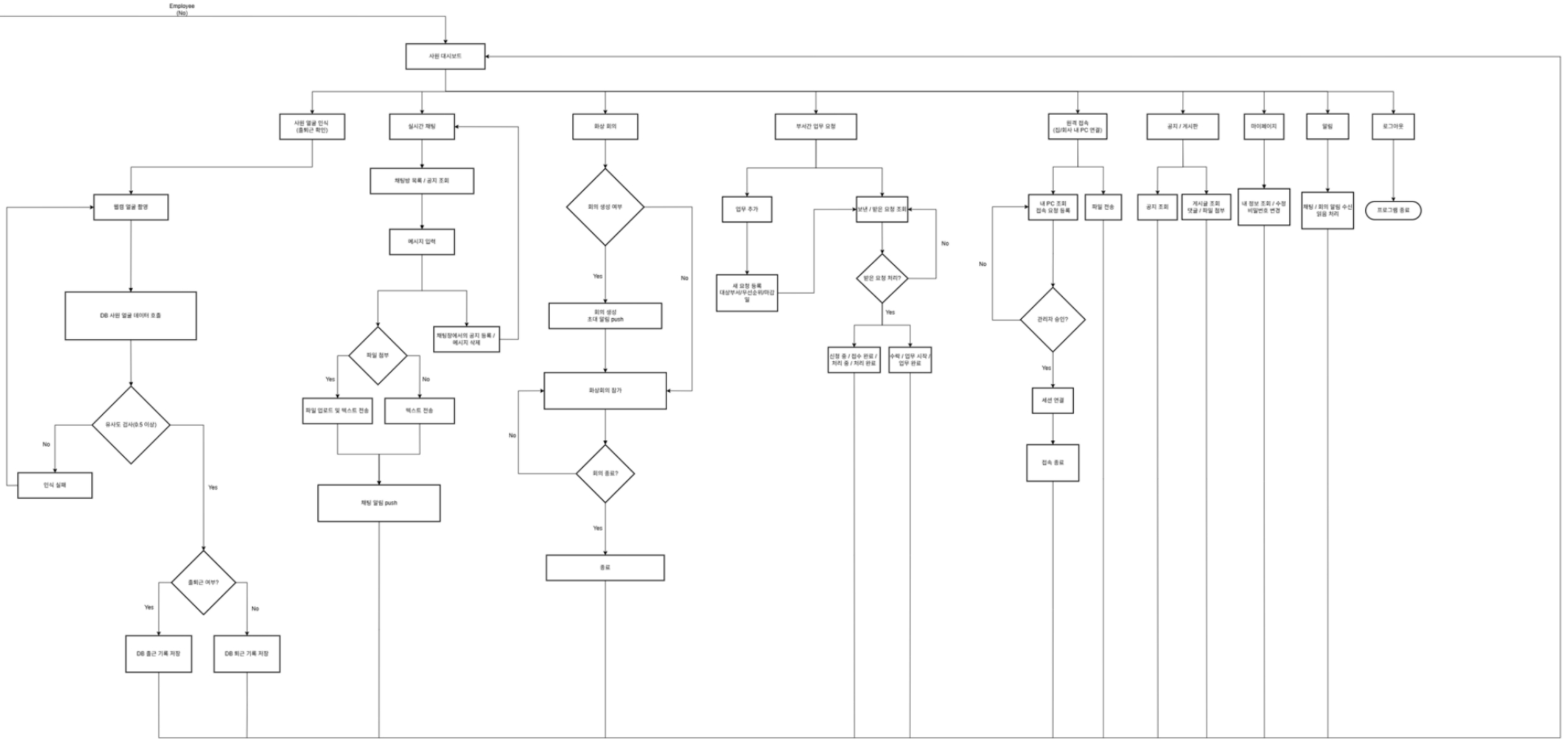
03. 순서도 - WEB


프로그램 시작
WEB
로그인 페이지
회원가입
로그인
이름/이메일/비밀번호
전화번호/부서/직위 입력
PC IP 자동 감지
사원번호 /
비밀번호 입력
Yes
No
이메일 중복?
인증 성공?
No
Yes
DB 회원 정보 및 QR 저장
사원번호 발급
Admin
(Yes)
권한 확인
Employee
(No)

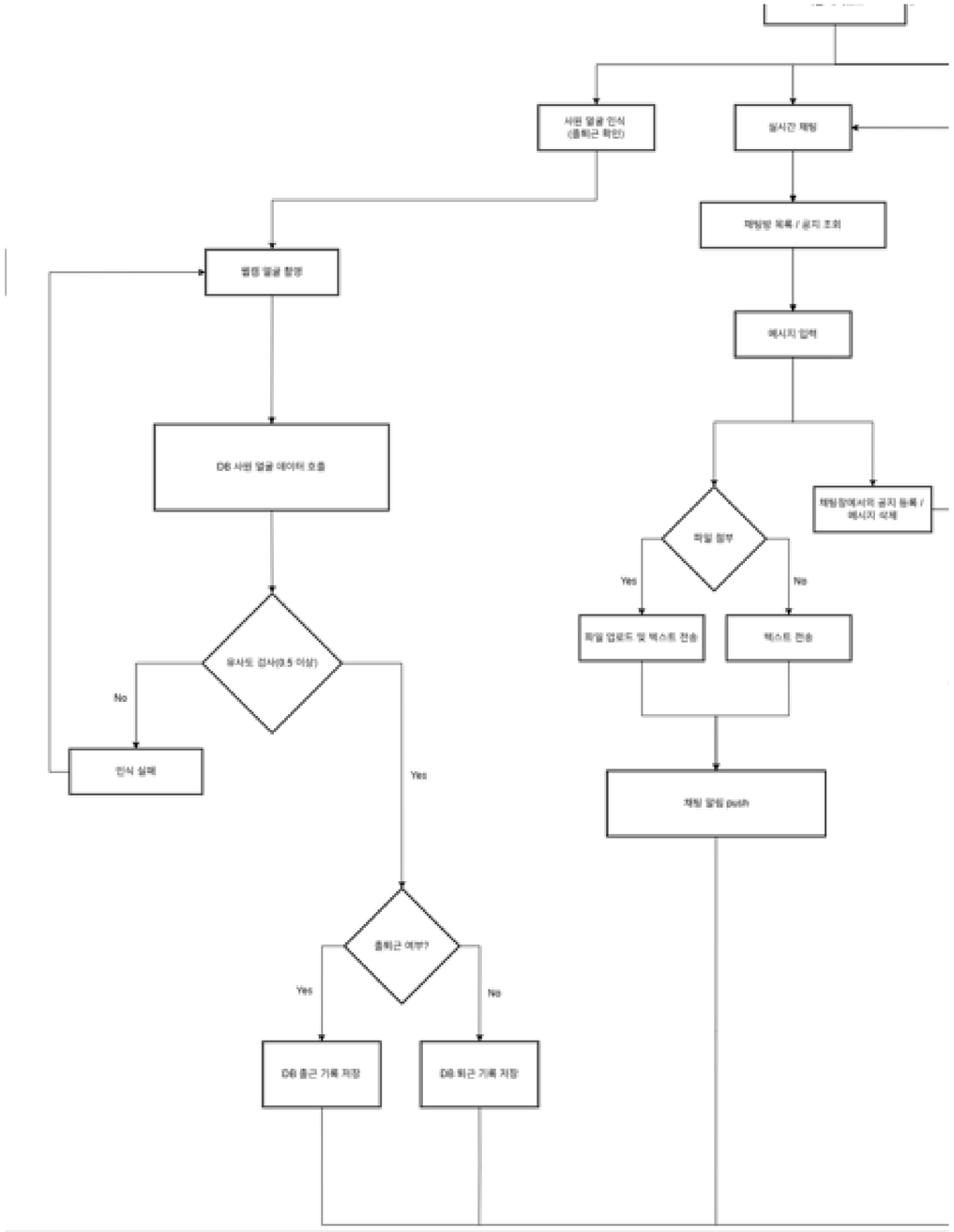
03. 순서도 - 관리자 대시보드



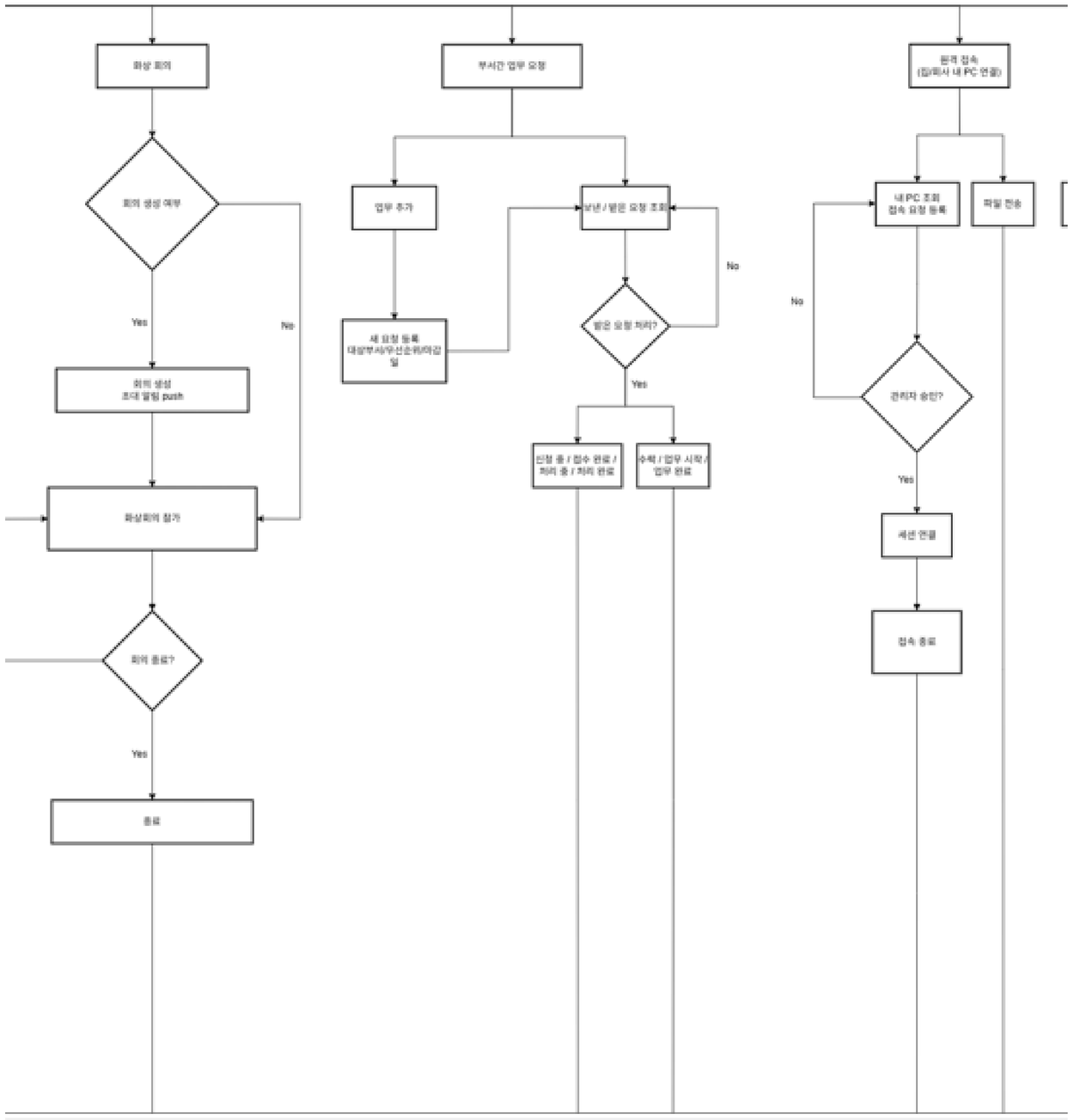
03. 순서도 - 사용자 대시보드



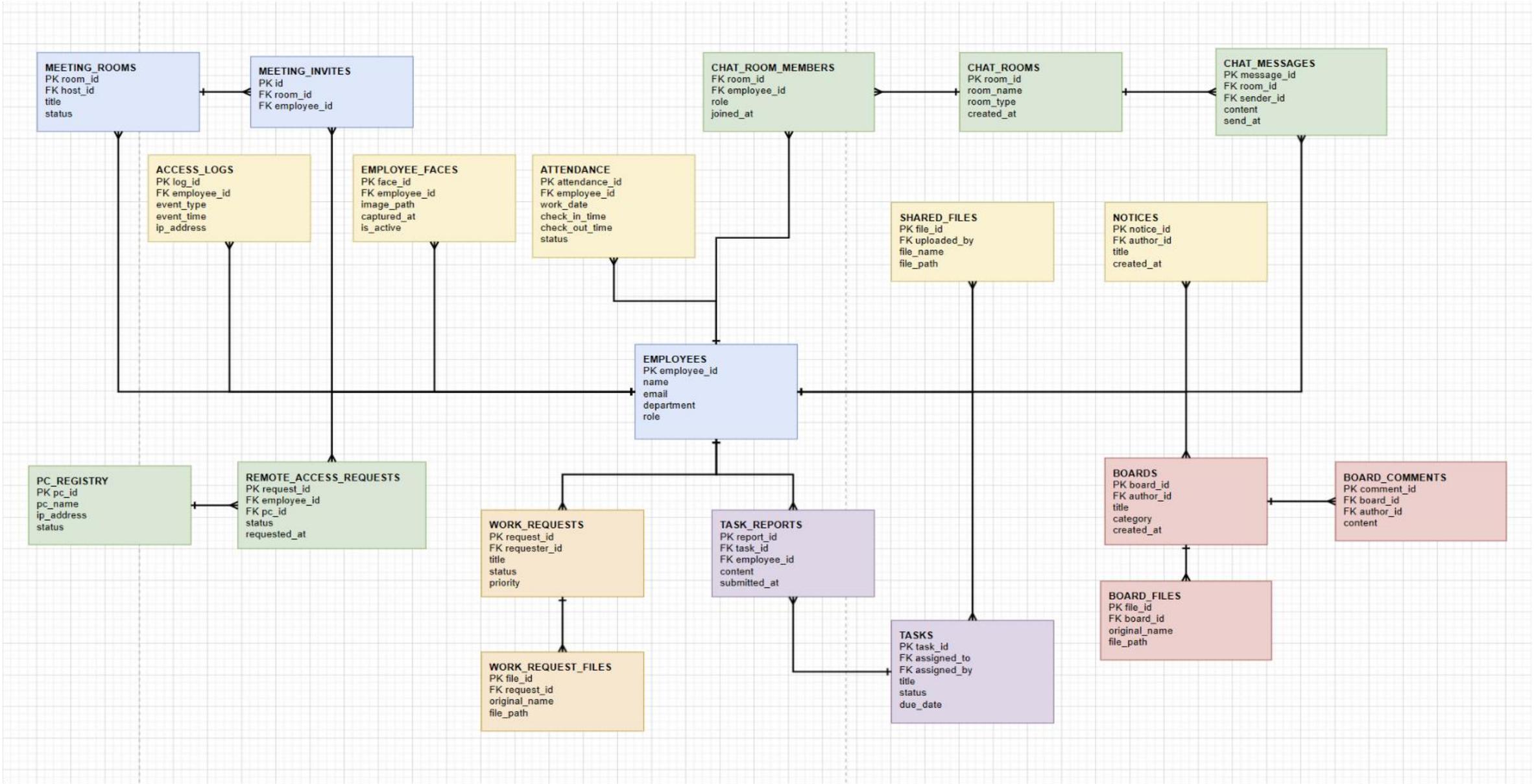
03. 순서도 - 사용자 대시보드



03. 순서도 - 사용자 대시보드



03. ERD


MEETING_ROOMS
PK room_id
FK host_id
title
status
MEETING_INVITES
PK id
FK room_id
FK employee_id
CHAT_ROOM_MEMBERS
FK room_id
FK employee_id
role
joined_at
CHAT_ROOMS
PK room_id
room_name
room_type
created_at
CHAT_MESSAGES
PK message_id
FK room_id
FK sender_id
content
send_at
ACCESS_LOGS
PK log_id
FK employee_id
event_type
event_time
ip_address
EMPLOYEE_FACES
PK face_id
FK employee_id
image_path
captured_at
is_active
ATTENDANCE
PK attendance_id
FK employee_id
work_date
check_in_time
check_out_time
status
SHARED_FILES
PK file_id
FK uploaded_by
file_name
file_path
NOTICES
PK notice_id
FK author_id
title
created_at
EMPLOYEES
PK employee_id
name
email
department
role
PC_REGISTRY
PK pc_id
pc_name
ip_address
status
REMOTE_ACCESS_REQUESTS
PK request_id
FK employee_id
FK pc_id
status
requested_at
WORK_REQUESTS
PK request_id
FK requester_id
title
status
priority
TASK_REPORTS
PK report_id
FK task_id
FK employee_id
content
submitted_at
BOARDS
PK board_id
FK author_id
title
category
created_at
BOARD_COMMENTS
PK comment_id
FK board_id
FK author_id
content
BOARD_FILES
PK file_id
FK board_id
original_name
file_path
TASKS
PK task_id
FK assigned_to
FK assigned_by
title
status
due_date
WORK_REQUEST_FILES
PK file_id
FK request_id
original_name
file_path

chaper 04.

프로젝트
수행 절차 및 방법

04. 프로젝트 수행절차 및 방법 - 개발 일정



04. 프로젝트 수행절차 및 방법 - 기술 스택 & 도구

개발 환경

- VS Code와 Visual Studio, 안드로이드 스튜디오를 활용해 웹과 하드웨어 개발 환경을 통합
- GitHub와 SourceTree를 통해 팀원들의 작업 병합 및 코드 이력 관리



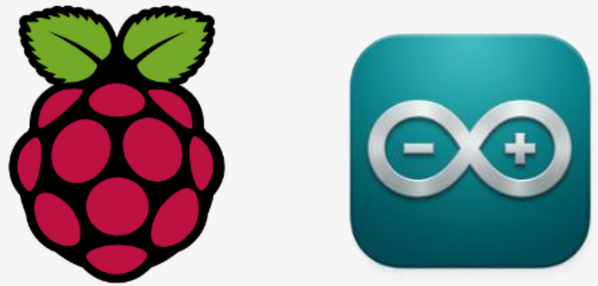
웹 기술 스택

- React를 이용해 사용자에게 빠르고 유연한 UI를 제공
- Flask 서버를 통해 데이터의 실시간 흐름을 제어
- MySQL로 근무자의 업무 상태와 기록을 체계적으로 구조화하고 안전하게 저장



하드웨어 제어

- 라즈베리파이를 통해 하드웨어를 연결하고 실시간 환경 제어 시스템 구현
- 아두이노를 통하여 라즈베리파이가 잡지 못하는 아날로그 신호를 제어



chaper 05.

프로젝트 결과

chaper 05-1.

사용자

03. 프로젝트 결과 - 업무관리

부서 간 업무 요청 기능



1. 같은 부서는 선택지에서 아예 제외 + 서버 이중 검증

```
// 내 부서는 선택지에서 제외
const myDept = currentUser?.department ||
setDepartments((data.departments || []).filter(d => d !== myDept))
```

department
디자인팀
운영팀
디자인팀
개발팀
개발팀
디자인팀
인사팀

```
if requester_dept == target_department:
    return jsonify({"success": False, "message": "같은 부서에는 요청할 수 없습니다."}), 400
```

→ 프론트엔드에서 같은 부서는
선택지에서 아예 표시되지 않고, 서버에서도 한 번 더 검증하여 같은
부서로의 업무 요청 API 우회까지 차단

개발팀 - 보낸 요청



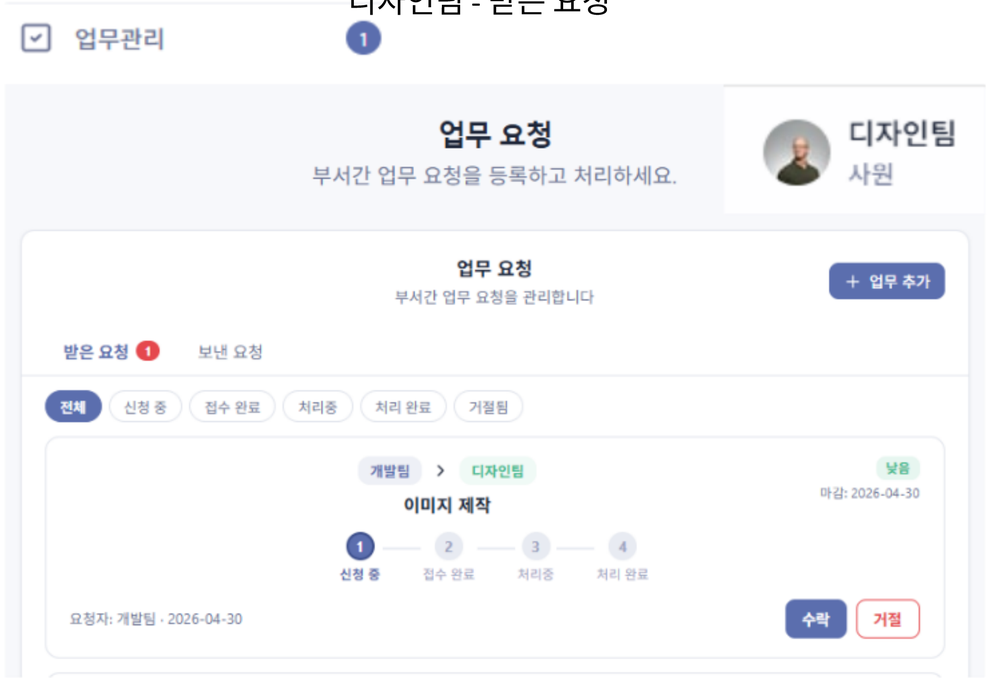
2. 받은 요청 / 보낸 요청 분리 조회

```
cur.execute("""
    SELECT wr.*, e.name AS requester_name
    FROM work_requests wr
    LEFT JOIN employees e ON wr.requester_id = e.employee_id
    WHERE wr.requester_department = %s
    ORDER BY wr.created_at DESC
""", (dept,))
```

→ 받은 요청과 보낸 요청을 탭으로 나눠 조회할 수 있어 업무 흐름을 명확하게 파악 가능

03. 프로젝트 결과 - 업무관리 (상태 관리 및 파일)

디자인팀 - 받은 요청



업무 요청 수락 후 처리시작



1. 상태 변경마다 시각 기록 / 수신 부서만 변경 가능

```
if dept and row["target_department"] != dept:
    return jsonify({"success": False, "message": "권한이 없습니다."}), 403

if new_status == "ACCEPTED":
    cur.execute("UPDATE work_requests SET status='ACCEPTED', accepted_at=NOW() WHERE request_id=%s", (req_id,))
elif new_status == "IN_PROGRESS":
    cur.execute("UPDATE work_requests SET status='IN_PROGRESS', started_at=NOW() WHERE request_id=%s", (req_id,))
elif new_status == "COMPLETED":
    cur.execute("UPDATE work_requests SET status='COMPLETED', completed_at=NOW() WHERE request_id=%s", (req_id,))
elif new_status == "REJECTED":
    cur.execute("""
        UPDATE work_requests SET status='REJECTED', rejected_at=NOW(), rejection_reason=%s
        WHERE request_id=%s
    """, (rejection_reason, req_id))
```

→ 상태가 변경될 때마다 시각을 자동으로 기록하며, 수신 부서만 상태를 변경할 수 있음.

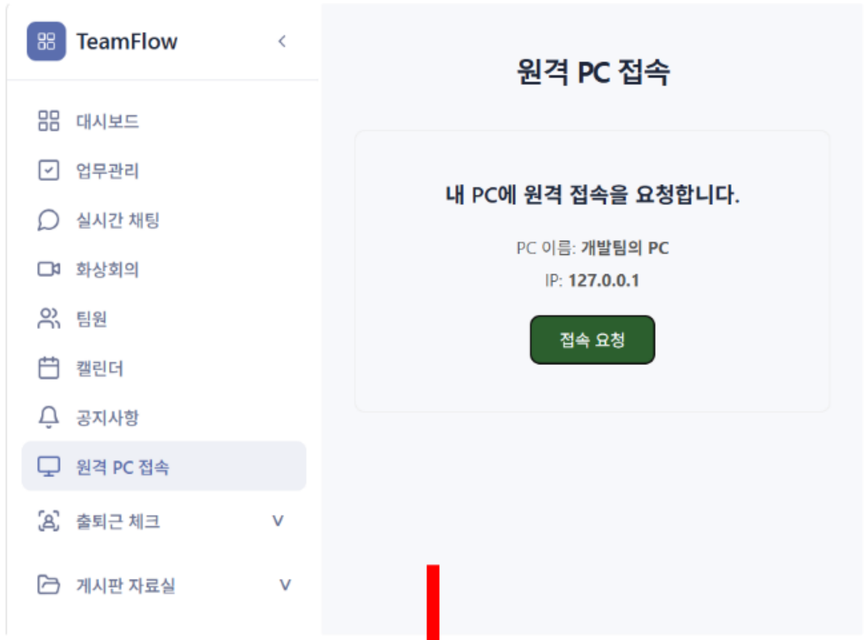
2. 파일 업로드 - 처리중 상태에서만 가능, 저장명 충돌 방지

```
def download_request_file(file_id):
    conn = None
    try:
        conn = get_conn()
        with conn.cursor() as cur:
            cur.execute("SELECT original_name, file_path FROM work_request_files WHERE file_id = %s", (file_id,))
            row = cur.fetchone()
        if not row:
            return jsonify({"success": False, "message": "파일을 찾을 수 없습니다."}), 404
        if not os.path.exists(row['file_path']):
            return jsonify({"success": False, "message": "서버에 파일이 없습니다."}), 404
        return send_file(row['file_path'], as_attachment=True, download_name=row['original_name'])
    except Exception as e:
        return jsonify({"success": False, "message": str(e)}), 500
    finally:
        if conn: conn.close()
```

→ 파일 업로드는 "처리중" 상태에서만 가능하도록 제한해 업무 흐름의 무결성을 유지.
동일한 파일명 충돌을 방지하기 위해 디스크에는 UUID 이름으로 저장

03. 프로젝트 결과 - 원격 PC접속 (요청 및 승인)

1. 사원이 PC 원격 접속 요청



2. 관리자 화면



1. 접속 요청 시 중복 요청 방지

```
existing = execute_query(
    "SELECT request_id FROM remote_access_requests " \
    "WHERE employee_id = %s AND pc_id = %s AND status = 'pending'",
    (employee_id, pc_id)
)
if existing:
    return jsonify({'request_id': existing[0]['request_id'], 'status': 'pending'})
```

→ 사원이 "원격 접속 요청" 버튼 클릭 -> 관리자 화면에 승인 요청
이때 중복 요청 방지 로직이 적용되어 이미 대기 중인 요청이 있으면 새 요청이 생성 X

2. 승인 여부를 3초마다 자동으로 확인

```
const poll = setInterval(async () => {
  try {
    const res = await axios.get(`/manage/remote/request/status?request_id=${requestId}`);
    if (res.data.status === 'approved') setApprovalState('approved');
    else if (res.data.status === 'rejected') setApprovalState('rejected');
  } catch {}
}, 3000);
```

→ 클라이언트는 3초마다 서버에 승인 상태를 폴링하여 자동으로 확인

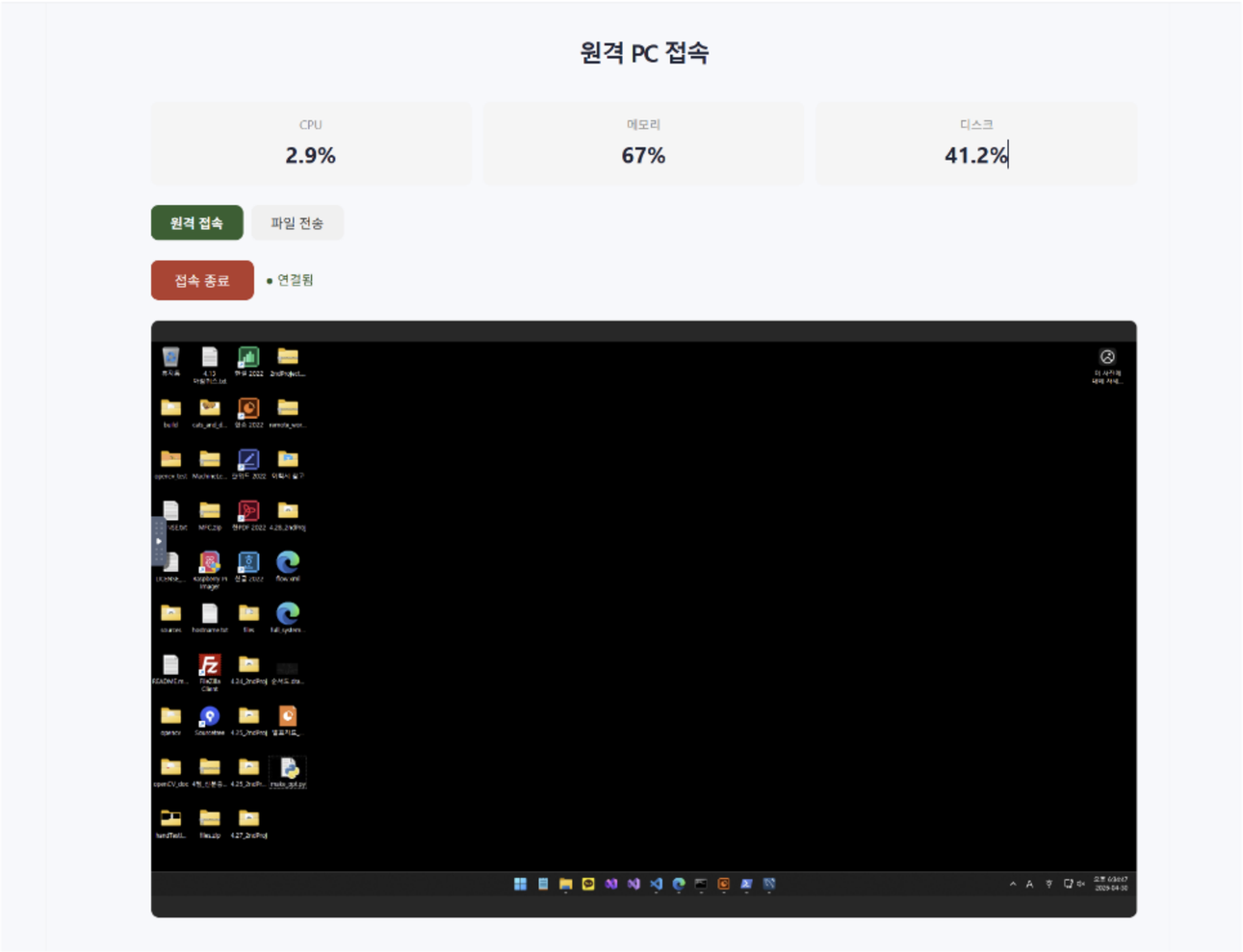
3. 관리자 승인/거절 처리

```
execute_query(
    "UPDATE remote_access_requests " \
    "SET status = %s, approved_by = %s, approved_at = %s " \
    "WHERE request_id = %s",
    (action, approved_by,
     datetime.now().strftime('%Y-%m-%d %H:%M:%S'), request_id),
```

→ 관리자가 승인 또는 거절 시 처리한 시각과 담당자를 함께 기록, 화면 자동 전환

03. 프로젝트 결과 - 원격 PC접속 (승인 후 결과)

3. 접속 승인 후 결과



1. 승인 후 브라우저에서 원격 화면 바로 표시

```
{status === "running" && novncUrl && (
  <iframe src={`${novncUrl}/vnc.html?autoconnect=true&resize=scale`}
  style={styles.iframe} title="Remote Desktop" />
)}
```

→ 관리자가 승인하면 서버에서 websockify를 실행하고,
React 화면에서 noVNC iframe으로 원격 PC 화면을 바로 표시.
별도 프로그램 설치 없이 브라우저에서 원격 접속이 가능

2. 드래그앤드롭 파일 업로드 / PC 파일 탐색 및 다운로드

```
// 업로드
const handleDrop = async (e) => {
  const formData = new FormData()
  formData.append('file', e.dataTransfer.files[0])
  await axios.post('/api/pc/upload', formData, { params: { path:
pcPath } })
}

// PC 파일 목록 조회
const fetchPcFiles = async (path) => {
  const res = await axios.get('/api/pc/files', { params: { path }
})
  setPcFiles(res.data.files)
}
```

→ 파일을 원격 PC에 업로드하거나, 원격 PC의 파일을 탐색하고 다운로드

03. 프로젝트 결과 - 원격 PC접속 (현황 관리)

4. 원격 접속 현황 관리



1. 현재 접속 중인 세션 목록 조회 / 접속 이력 분리 표시

```
rows = execute_query('''
  SELECT e.name, p.pc_name, p.ip_address,
         r.approved_at, r.disconnected_at
  FROM   remote_access_requests r
  JOIN   employees    e ON r.employee_id = e.employee_id
  JOIN   pc_registry p ON r.pc_id = p.pc_id
  WHERE  r.status = 'approved'
  ORDER  BY r.approved_at DESC
''')
```

```
const active   = sessions.filter(s => !s.disconnected_at)
const inactive = sessions.filter(s =>  s.disconnected_at)
```

→ 사원명과 PC명을 JOIN한 테이블로 현재 접속 중인 세션 목록을 조회하며, disconnected_at 유무에 따라 "현재 접속 중"과 "접속 이력"을 두 테이블로 분리하여 표시

2. 관리자 강제 종료 → 사용자 화면 자동 초기화

```
execute_query(
    "UPDATE remote_access_requests SET disconnected_at = %s WHERE request_id = %s",
    (datetime.now().strftime('%Y-%m-%d %H:%M:%S'), request_id),
    fetch=False
)
```

```
if (res.data.disconnected_at && !selfStopped.current) {
  clearInterval(poll);
  setStatus("stopped");
  setApprovalState('idle');
  setRequestId(null);
  alert('관리자에 의해 원격 접속이 종료되었습니다.');
}
```

→ 관리자가 종료하면 안내 메시지 표시, 5초 이내에 사용자 화면이 자동으로 초기 상태로 돌아감

03. 프로젝트 결과 - 사용자 UI_실시간 채팅_채팅창



- 로그인한 사용자가 참여 중인 채팅방을 조회하고, 선택한 채팅방의 메시지를 1초마다 자동 갱신하여 실시간 채팅처럼 동작하도록 구현

- 채팅방 ID를 기준으로 저장된 데이터를 조회하고, 삭제되지 않은 메시지, 공지, 사진 및 동영상 파일만 사용자 화면에 전달

1. 1초마다 채팅을 갱신

```
useEffect(() => {
  if (!activeChannelId || !currentUserId) return
  fetchRoomNotice(activeChannelId)
  fetchMessages(activeChannelId)

  const timer = setInterval(() => {fetchMessages(activeChannelId)
  fetchRoomNotice(activeChannelId)}, 1000)

  return () => clearInterval(timer)
}, [activeChannelId, currentUserId])
```

2. DB에서 데이터 조회 후 채팅 화면에 띄움

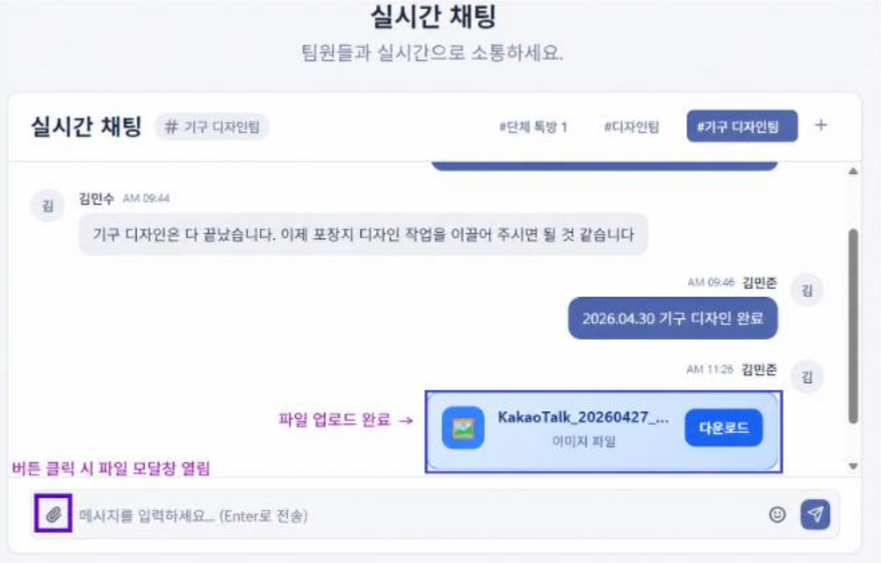
```
@chat_bp.route("/messages/<int:room_id>", methods=["GET"])
def get_messages(room_id):
    conn = None

    try:
        conn = get_conn()

        with conn.cursor() as cur:
            cur.execute(
                """
                SELECT
                    cm.message_id,
                    cm.room_id,
                    cm.sender_id,
                    e.name AS sender_name,
                    e.position AS sender_position,
                    '' AS sender_avatar,
                    cm.content,
                    cm.message_type,
                    cm.file_name,
                    cm.file_path,
                    cm.file_size,
                    cm.mime_type,
                    cm.is_notice,
                    DATE_FORMAT(cm.send_at, '%%p %%h:%%i') AS time,
                    cm.send_at
                FROM chat_messages cm
                LEFT JOIN employees e
                    ON cm.sender_id = e.employee_id
                WHERE cm.room_id = %s
                  AND IFNULL(cm.is_deleted, 0) = 0
                ORDER BY cm.message_id ASC
                """,
                (room_id,)
            )
```

03. 프로젝트 결과 - 사용자 UI_실시간 채팅_파일·공지 등록

-파일 등록



1. 선택 파일을 FormData 형태로 서버에 전송 후 창에 파일 표시

```
const formData = new FormData()
formData.append('room_id', activeChannelId)
formData.append('sender_id', currentUserId)
formData.append('file', file)
const res = await fetch(`${API_BASE}/api/chat/files`, {
  method: 'POST',
  body: formData,
})
```

2. 업로드 파일을 서버 폴더에 저장 후 DB에 FILE 타입 메세지로 기록

```
@chat_bp.route("/files", methods=["POST"])
def upload_chat_file():
    file.save(save_path)
    cur.execute(
        """
        INSERT INTO chat_messages (
            room_id,sender_id,content, message_type, file_name, file_path,
            file_size, mime_type, is_deleted, is_notice, send_at
        )
        VALUES (%s, %s, %s, 'FILE', %s, %s, %s, %s, 0, 0, NOW())
        """,
        (
            room_id, sender_id, original_name, original_name,
            saved_name,file_size, mime_type
        )
    )
```

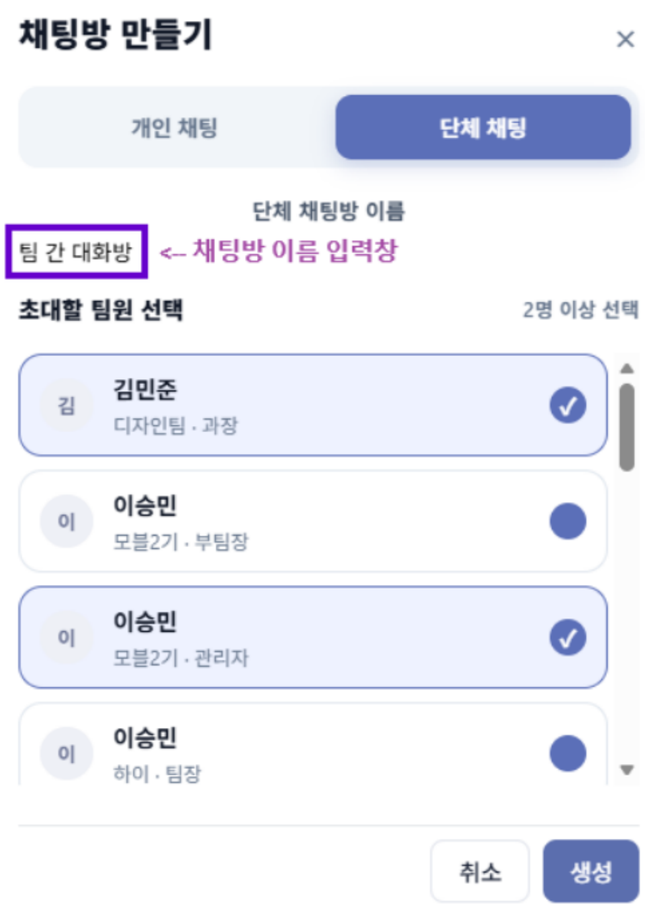
-공지 등록

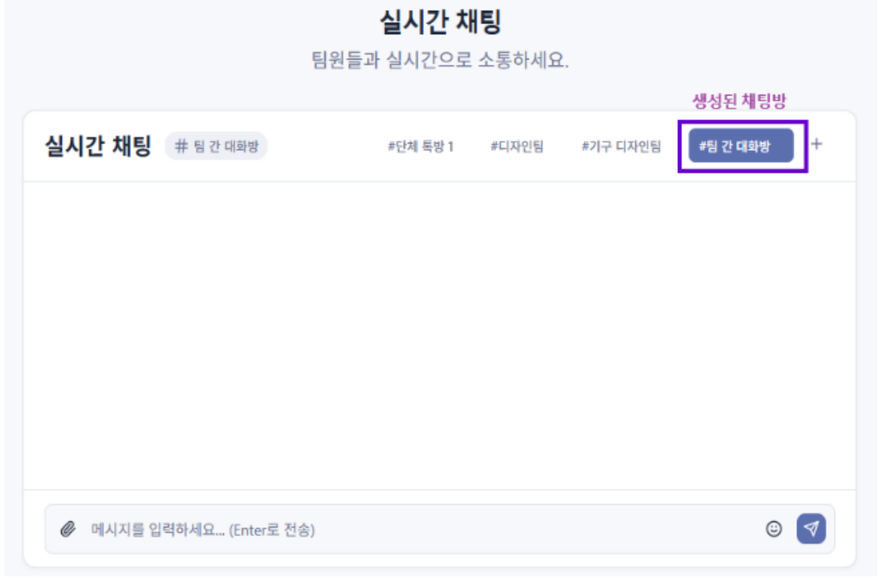


3. 선택한 메시지를 공지로 지정하여 채팅방 상단에 고정 표시

```
cur.execute(
    """
    UPDATE chat_messages
    SET is_notice = 1
    WHERE room_id = %s
      AND message_id = %s
    """,
    (room_id, message_id)
)
```

03. 프로젝트 결과 - 사용자 UI_실시간 채팅_채팅방 생성





1. 사용자가 정한 정보를 Flask 서버에 전송

```
const res = await fetch(`${API_BASE}/api/chat/rooms`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    room_type: newRoomType === 'private' ? 'DIRECT' : 'GROUP',
    room_name: roomName,
    creator_id: currentUserId,
    member_ids: memberIds,
  }),
})
```

2. 채팅방 정보를 chat_rooms에 저장하고, 선택한 팀원 들을 chat_room_members에 참여자로 등록

```
@chat_bp.route("/rooms", methods=["POST"])
def create_room():
    data = request.get_json(silent=True) or {}

    room_type = (data.get("room_type") or "GROUP").upper()
    room_name = (data.get("room_name") or "").strip()
    creator_id = data.get("creator_id")
    member_ids = data.get("member_ids") or []

cur.execute(
    """
    INSERT INTO chat_rooms (room_type, room_name, deleted_id, created_at)
    VALUES (%s, %s, 0, NOW())
    """,
    (room_type, room_name)
)
room_id = cur.lastrowid

for member_id in member_ids:
    cur.execute(
        """
        INSERT INTO chat_room_members (room_id, employee_id, joined_at)
        VALUES (%s, %s, NOW())
        """,
        (room_id, member_id)
    )
```

03. 프로젝트 결과 - 회상회의



① Socket.IO로 방 입장 신호 전송
② 서버가 기존 참여자 목록 전달
③ 참여자마다 WebRTC offer/answer 교환 → P2P 연결 수립
④ 연결 완료 → 영상 스트림 직접 수신

```
/* 소켓 연결 */
socket = io(API, { transports: ['websocket', 'polling'] });
socketRef.current = socket;

socket.on('connect', () => {
    if (cancelled) return;
    setSocketStatus('connected');
    socket.emit('join-room', {
        room_id: room.room_id,
        user_id: user.employee_id,
        user_name: user.name || '참여자',
    });
});
```

```
@socketio.on("join-room")
def handle_join(data):
    room_id = str(data.get("room_id", ""))
    user_id = data.get("user_id")
    user_name = data.get("user_name", "참여자")
    sid = request.sid

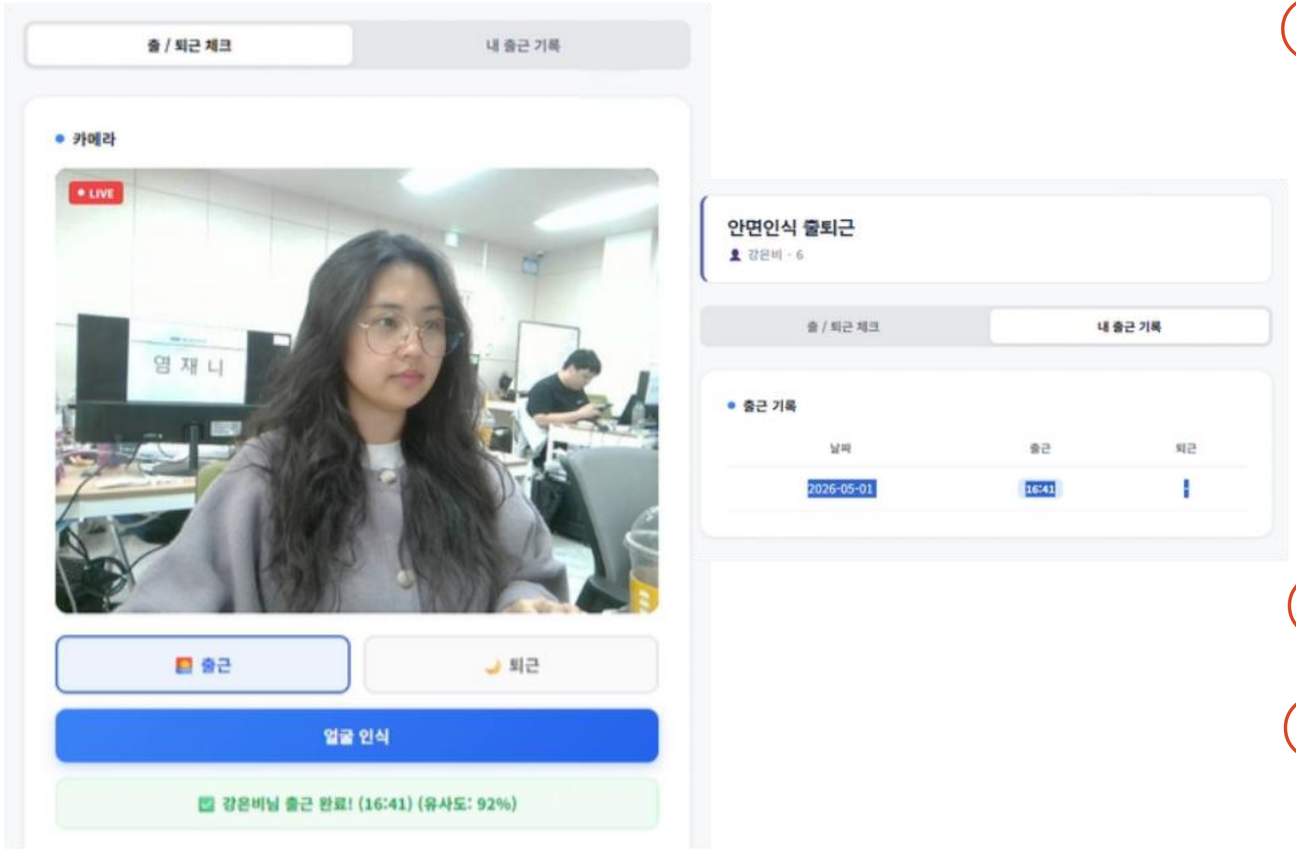
    existing = [
        {
            "socket_id": s,
            "user_id": info["user_id"],
            "user_name": info["user_name"]
        }
        for s, info in room_users[room_id].items()
    ]
    emit("room-users", existing)
    socketio.emit(
        "user-joined",
        {
            "socket_id": sid,
            "user_id": user_id,
            "user_name": user_name
        },
        room=room_id,
        skip_sid=sid
    )
```

```
socket.on('room-users', async (users) => {
    if (cancelled) return;
    for (const u of users) {
        peerNamesRef.current[u.socket_id] = { user_name: u.user_name, user_id: u.user_id };
        setPeers(prev => ({
            ...prev,
            [u.socket_id]: { stream: null, user_name: u.user_name, user_id: u.user_id },
        }));
        const pc    = createPeerConn(u.socket_id);
        const offer = await pc.createOffer();
        await pc.setLocalDescription(offer);
        socket.emit('offer', { to: u.socket_id, offer });
    }
});
```

```
socket.on('offer', async ({ from, offer }) => {
    if (cancelled) return;
    const info = peerNamesRef.current[from] || {};
    setPeers(prev => prev[from] ? prev : {
        ...prev,
        [from]: { stream: null, user_name: info.user_name || '참여자', user_id: info.user_id },
    });
    const pc = createPeerConn(from);
    await pc.setRemoteDescription(new RTCSessionDescription(offer));
    const answer = await pc.createAnswer();
    await pc.setLocalDescription(answer);
    socket.emit('answer', { to: from, answer });
});
```

```
pc.ontrack = ({ streams }) => {
    setPeers(prev => ({
        ...prev,
        [sid]: { ...prev[sid], stream: streams[0] },
    }));
};
```

03. 프로젝트 결과 - 출퇴근 체크_안면인식



① 서버 카메라 영상을 브라우저에 실시간 표시 (MJPEG 스트림)
② 버튼 클릭 → 현재 프레임 1장을 서버에서 가져옴 (스냅샷)
③ 스냅샷을 Flask로 전송 → InsightFace가 얼굴 벡터(512차원) 추출
④ 등록된 직원 벡터와 코사인 유사도 비교 → 가장 높은 사람 선택
⑤ 유사도 > 50% 이고 본인 ID와 일치하면 → DB에 출/퇴근 기록

② 버튼 클릭 → 현재 프레임 1장을 서버에서 가져옴 (스냅샷)

```
const captureSnapshot = async () => {
    const res = await fetch(`${API}/api/camera/feed/snapshot?t=${Date.now()}`);
    if (!res.ok) throw new Error('스냅샷 요청 실패');
    return await res.blob();
};

const handleVerify = async () => {
    if (!mode) return alert('출근 또는 퇴근을 먼저 선택하세요!');

    const check = await axios.get(`${API}/api/face/check-registered?employee_id=${user.employee_id}`);
    if (!check.data.registered) {
        alert('등록된 얼굴이 없습니다. 얼굴 등록 먼저 해주세요!');
        return;
    }

    setIsVerifying(true);
    setStatus('얼굴 인식 중...');

    try {
        const imageBlob = await captureSnapshot();
        const formData = new FormData();
        formData.append('image', imageBlob, 'snapshot.jpg');
        formData.append('employee_id', user.employee_id);

        const verify = await axios.post(`${API}/api/face/verify`, formData, {
            headers: { 'Content-Type': 'multipart/form-data' }
        });
```

③ 스냅샷을 Flask로 전송 → InsightFace가 얼굴 벡터(512차원) 추출

```
face_app = FaceAnalysis(name='buffalo_l', providers=['CPUExecutionProvider'])
face_app.prepare(ctx_id=0, det_size=(160, 160))
```

④ 등록된 직원 벡터와 코사인 유사도 비교 → 가장 높은 사람 선택

```
def cosine_similarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

for i, known_emb in enumerate(known_embeddings):
    score = cosine_similarity(embedding, known_emb)
    print(f"[DEBUG] {known_ids[i]} 유사도: {score:.4f}")
    if score > best_score:
        best_score = score
        best_id = known_ids[i]
```

⑤ 유사도 > 50% 이고 본인 ID와 일치하면 → DB에 출/퇴근 기록

```
if best_score < THRESHOLD or best_id is None:
    return jsonify({"result": "fail", "message": f"얼굴 인식 실패 (유사도: {best_score:.2f})"})

if best_id != session_employee_id:
    return jsonify({"result": "fail", "message": "본인 얼굴이 아닙니다."})
```

03. 프로젝트 결과 - 사용자 UI_QR 인식 기반 출퇴근 기능 - 1











03. 프로젝트 결과 - 사용자 UI_QR 인식 기반 출퇴근 기능 - 코드

1. 사용자가 출근/퇴근을 선택한 뒤 QR 인식 버튼을 누르면
   Flask 서버로 인식 요청을 보내고 결과를 화면에 표시

```
const handleQrScan = async () => {
    const res = await axios.post(`${API_BASE}/api/attendance/qr-scan`, {
        mode,
    })

    const data = res.data

    if (data.result === 'success' || data.success === true) {
        setStatus(`☑ ${data.employee?.name || '직원'}님 ${data.message}`)
    } else {
        setStatus(`✕ ${data.message || 'QR 인식에 실패했습니다.'}`)
    }
```

2. Flask 카메라 스트리밍 API를 사용자 화면에 연결하여
   실시간 QR 인식 화면을 제공

```
<img
    src={`${API_BASE}/api/camera/feed`}
    alt="qr-camera"
    style={S.camImg}
/>
```

3. 카메라 프레임에서 QR을 인식하고 직원 ID를 추출하여 출근/퇴근 기록을 DB에 저장

```
@qr_attendance_bp.route("/api/attendance/qr-scan", methods=["POST"])
def qr_scan_attendance():

    frame = get_latest_frame()

    qr_text = decode_qr_from_frame(frame)

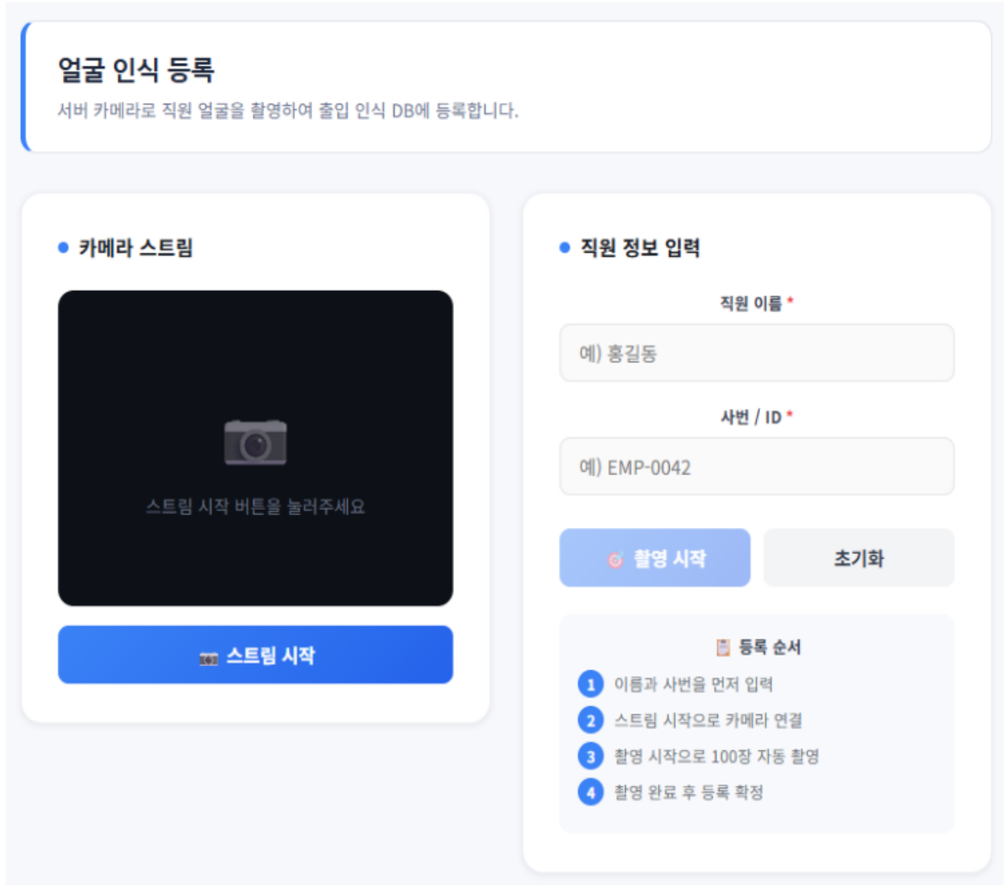
    employee = find_employee_by_qr(qr_text)

    if mode == "출근":
        result = check_in_by_qr(employee["employee_id"])
    else:
        result = check_out_by_qr(employee["employee_id"])
```

chaper 05-2.

관리자

03. 프로젝트 결과 - 직원목록_얼굴등록



① 웹캠 프레임에서 얼굴 영역 감지 하여 100장 저장
② 직원 폴더에 저장
③ 인식용 데이터 분석 시작
(별도의 스레드로 실행하여 아래 코드의 실행이 끊기지 않음)
④ 촬영 진행률을 화면에 표시하고, 프레임을 JPEG으로 압축해 웹으로 스트리밍

```python
def generate_register_frames():
    while True:
        frame = get_latest_frame()
        if frame is None:
            time.sleep(0.033)
            continue

        ① if cap_state['capturing'] and cap_state['count'] < CAPTURE_MAX:
            gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
            faces = face_cascade.detectMultiScale(gray, 1.3, 5)

            for (x, y, w, h) in faces:
                cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)

                ② save_dir = f"employees/{cap_state['employee_id']}"
                os.makedirs(save_dir, exist_ok=True)
                img_path = f"{save_dir}/{cap_state['count']:03d}.jpg"
                cv2.imwrite(img_path, frame)
                cap_state['count'] += 1

                if cap_state['count'] >= CAPTURE_MAX:
                    cap_state['capturing'] = False
                    ③
                    _reload_employees()

        ④
        label = f"Capturing: {cap_state['count']}/{CAPTURE_MAX}"
        cv2.putText(frame, label, (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)

        _, buffer = cv2.imencode('.jpg', frame, [cv2.IMWRITE_JPEG_QUALITY, 70])
        yield (b'--frame\r\nContent-Type: image/jpeg\r\n\r\n' + buffer.tobytes() + b'\r\n')
        time.sleep(0.033)
```

03. 프로젝트 결과 - 직원목록_얼굴등록

①
```python
def _reload_employees():
    def _run():
        try:
            from FaceReco.recognize import rebuild_employees
            rebuild_employees()
            print("[INFO] 얼굴 임베딩 DB 재빌드 완료")
        except Exception as e:
            print(f"[WARN] 얼굴 임베딩 재빌드 실패: {e}")
    threading.Thread(target=_run, daemon=True).start()
```

②
```python
def rebuild_employees():
    if os.path.exists(CACHE_FILE):
        os.remove(CACHE_FILE)
        print("[INFO] 임베딩 캐시 삭제됨 - 재빌드 시작")
    _build_embeddings()
```

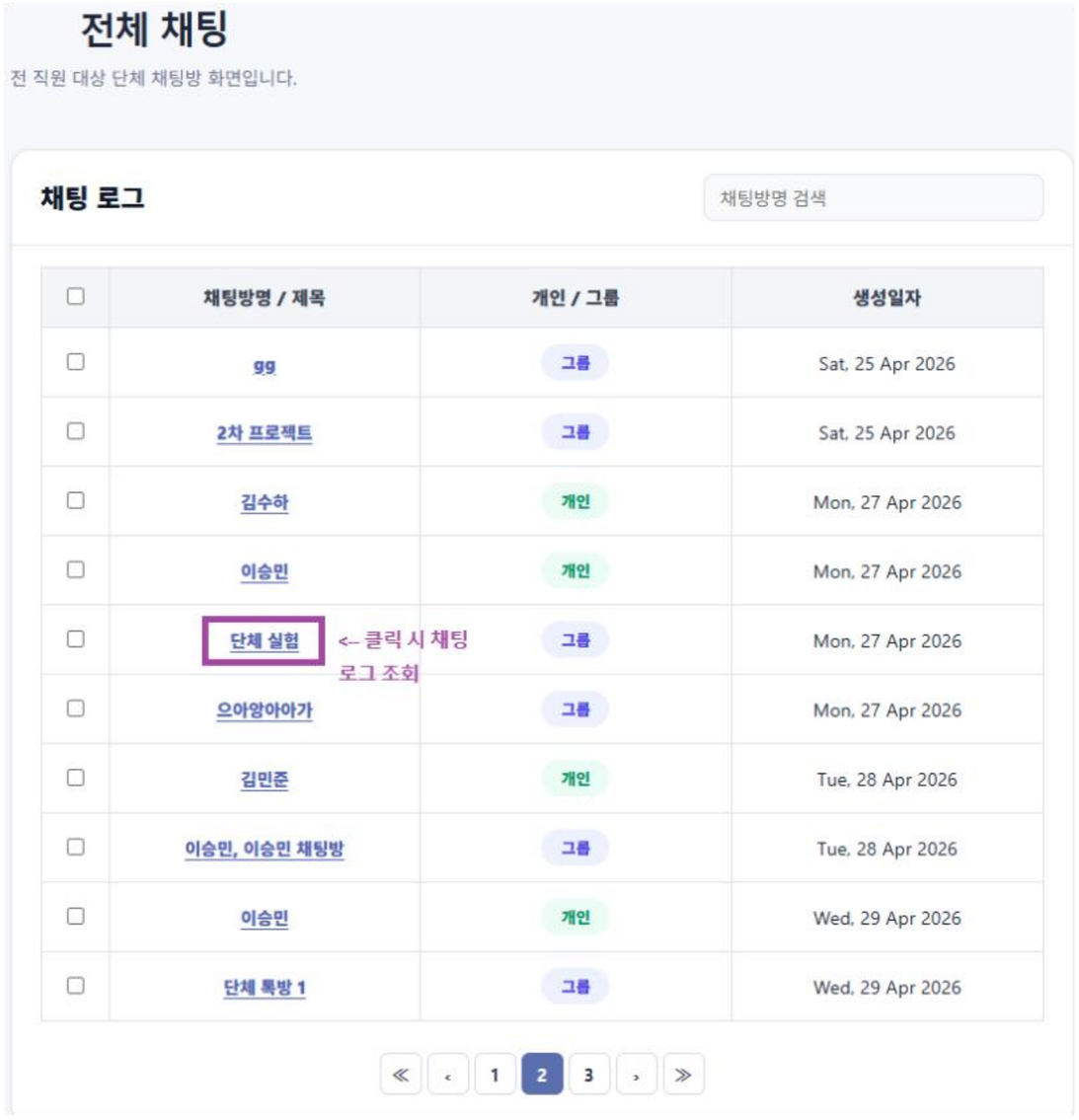
③
```python
face_app = FaceAnalysis(name='buffalo_l', providers=['CPUExecutionProvider'])
face_app.prepare(ctx_id=0, det_size=(160, 160))
```

④
```python
def _build_embeddings():
    global known_embeddings, known_ids
    known_embeddings = []
    known_ids = []
    emp_dir = "employees"
    if not os.path.exists(emp_dir):
        print("[WARN] employees 디렉토리가 없습니다.")
        return
    for emp_id in os.listdir(emp_dir):
        person_dir = os.path.join(emp_dir, emp_id)
        if not os.path.isdir(person_dir):
            continue
        embeddings = []
        for img_file in os.listdir(person_dir):
            if not img_file.endswith('.jpg'):
                continue
            img = cv2.imread(os.path.join(person_dir, img_file))
            if img is None:
                continue
            faces = face_app.get(img)
            if faces:
                embeddings.append(faces[0].embedding)
        if embeddings:
            known_embeddings.append(np.mean(embeddings, axis=0))
            known_ids.append(emp_id)

    if known_ids:
        np.savez(CACHE_FILE, embeddings=known_embeddings, ids=known_ids)
    print(f"[INFO] 총 {len(known_ids)}명 임베딩 계산 및 캐시 저장 완료: {known_ids}")
```

① 별도 스레드로 실행하여 서버 응답 유지
② 기존 캐시 삭제 후 임베딩 전체 재계산
③ buffalo_l 모델로 직원 전체의 사진 100장씩 분석 (1~3분 소요)
④ 100장 평균값으로 대표 임베딩 1개 생성

03. 프로젝트 결과 - 관리자-> 전체 채팅 로그





- 관리자 화면에서 전체 채팅방 목록을 조회하고, 채팅방 제목을 클릭하면 해당 채팅방의 메시지 로그를 읽기 전용으로 확인할 수 있도록 구현

- 채팅방의 유형이 개인 또는 그룹인지를 확인 가능

03. 프로젝트 결과 - 관리자-> 전체 채팅 로그 - 코드

1. 전체 채팅방 목록 조회

```
async function loadChatRooms() {
  const data = await apiGet('/api/chatrooms')
  const rooms = normalizeList(data, 'chatrooms')

  const converted = rooms.map((room) => {
    const rawType =
      room.room_type ||
      room.type ||
      room.roomType ||
      room.chat_type ||
      room.chatType ||
      'GROUP'

    return {
        id: room.room_id || room.id,
        name: room.room_name || room.name || `채팅방 ${room.room_id || room.id}`,
        type: normalizeRoomType(rawType),
        createdAt: room.created_at || room.createdAt || '',
        deletedId: room.deleted_id ?? 0,
        unread: 0,
    }
  }).filter((room) => room.id && Number(room.deletedId) === 0)

  setChannels(converted)
```

2. 한 페이지당 10개씩 나누어 표시

```
const ROOM_PAGE_SIZE = 10

const pagedChannels = channels.slice(
    (safeRoomPage - 1) * ROOM_PAGE_SIZE,
    safeRoomPage * ROOM_PAGE_SIZE
)
```

3. 채팅방 제목 클릭 시 채팅 로그를 조회

```
async function handleRoomTitleClick(room) {

        setSelectedRoom(room)
        setDetailOpen(true)

        const data = await apiGet(`/api/chatlogs?room_id=${room.id}`)
        const rows = normalizeList(data, 'chatlogs')

        const converted = rows.map((log) => ({
            id: log.id || log.message_id || log.chat_id,
            roomId: log.room_id,
            senderId: log.sender_id,
            userName: log.user || log.name || log.sender_name || String(log.sender_id || '알 수 없음'),
            content: log.message || log.content || '',
            time: log.time || log.send_at || '',
            messageType: log.message_type || 'TEXT',
            fileName: log.file_name || '',
            fileUrl: log.file_url ? `${FILE_BASE}${log.file_url}` : '',
            mimeType: log.mime_type || '',
            isNotice: Number(log.is_notice || 0) === 1,
        }))

        setDetailMessages(converted)
```

4. 선택한 채팅방의 메시지와 작성자 정보를 조회하여 관리자 화면에 채팅 로그로 전달

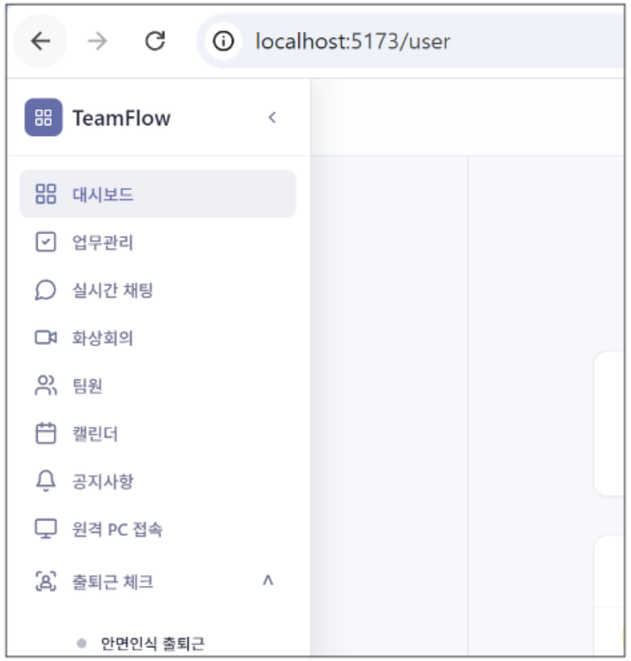
```
@Mchat_bp.route("/app/chatlogs", methods=["GET"])
def get_chatlogs():
    conn = None
    try:
        room_id = request.args.get("room_id", "1")

        conn = get_conn()

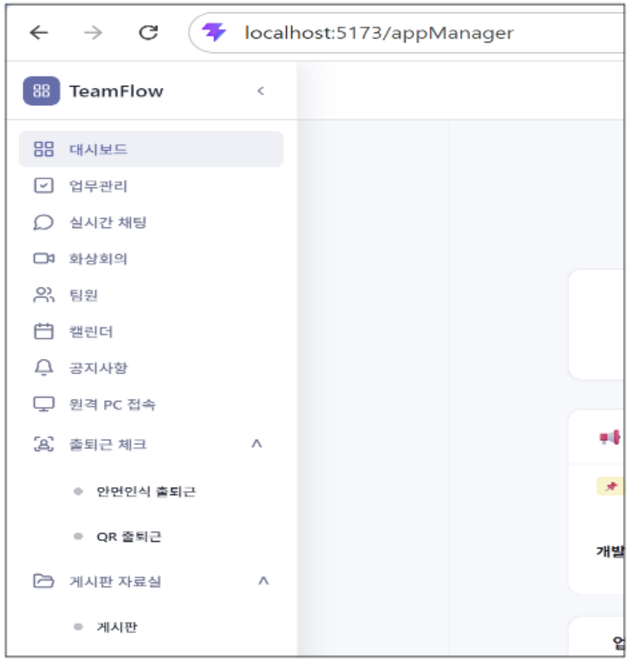
        with conn.cursor() as cur:
            cur.execute(
                """
                SELECT
                    cm.message_id AS id, cm.message_id, cm.room_id, cm.sender_id,
                    cm.content AS message, cm.is_deleted, cm.send_at AS time, e.name AS user
                FROM chat_messages cm
                LEFT JOIN employees e
                    ON cm.sender_id = e.employee_id
                WHERE cm.room_id = %s
                  AND cm.is_deleted = 0
                ORDER BY cm.send_at ASC, cm.message_id ASC
                """,
                (room_id,)
            )
```

03. 프로젝트 결과 - 사용자 권한별 페이지 접근 제어

직원 페이지



관리자 페이지




```
useEffect(() => {
    const token = getStoredToken()
    const role = getLoginRole()

    if (!token || isTokenExpired()) {
        clearAuthStorage()
        window.location.replace('/')
        return
    }

    if (role !== 'ADMIN') {
        window.location.replace('/user')
        return
    }

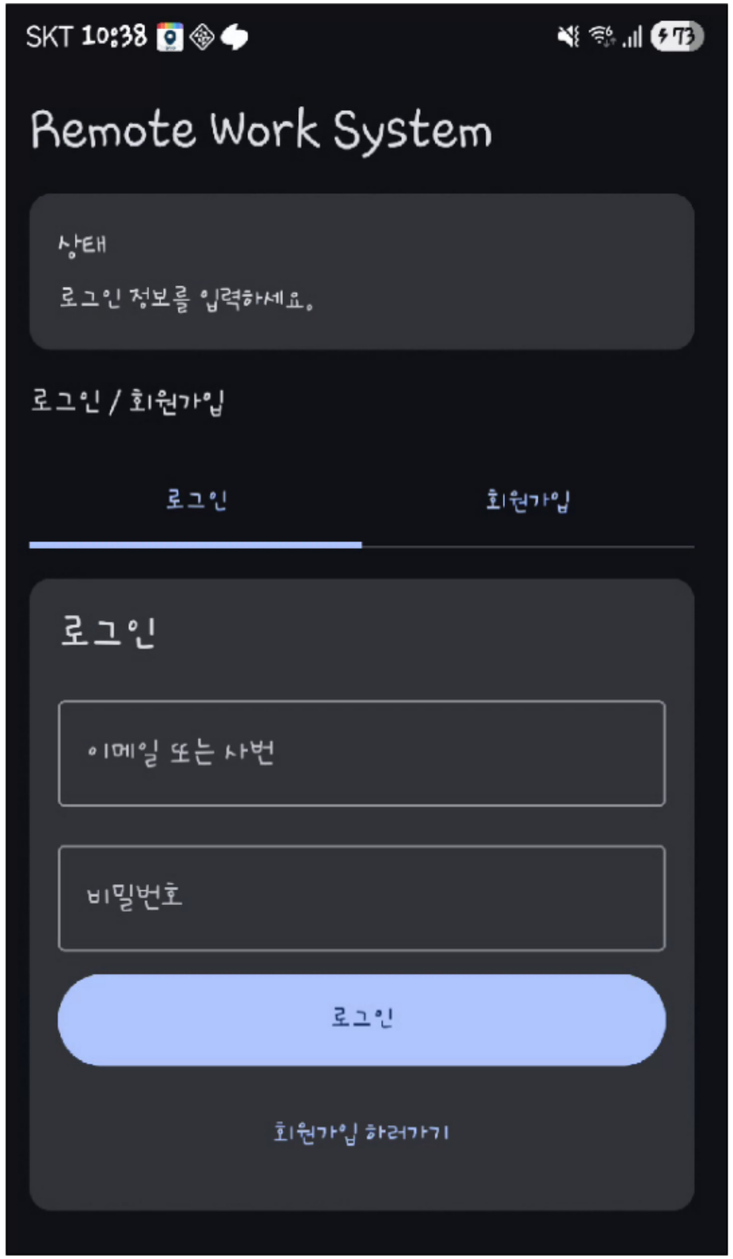
    setAuthChecked(true)
}, [])
```

- 일반 사용자는 직원 전용 페이지(/user)만 접근 가능
- 토큰이 없거나 만료된 경우 로그인페이지로 돌아가게 구현
- 관리자 외 유저가 주소창에 강제로 관리자 페이지 접속할경우 다시 사용자 전용 페이지로 반환

chaper 05-3.

Application

03. 프로젝트 결과 - 직원 사원증 - 로그인




```
user = execute_query(sql, (user_id,))
employee = user[0]

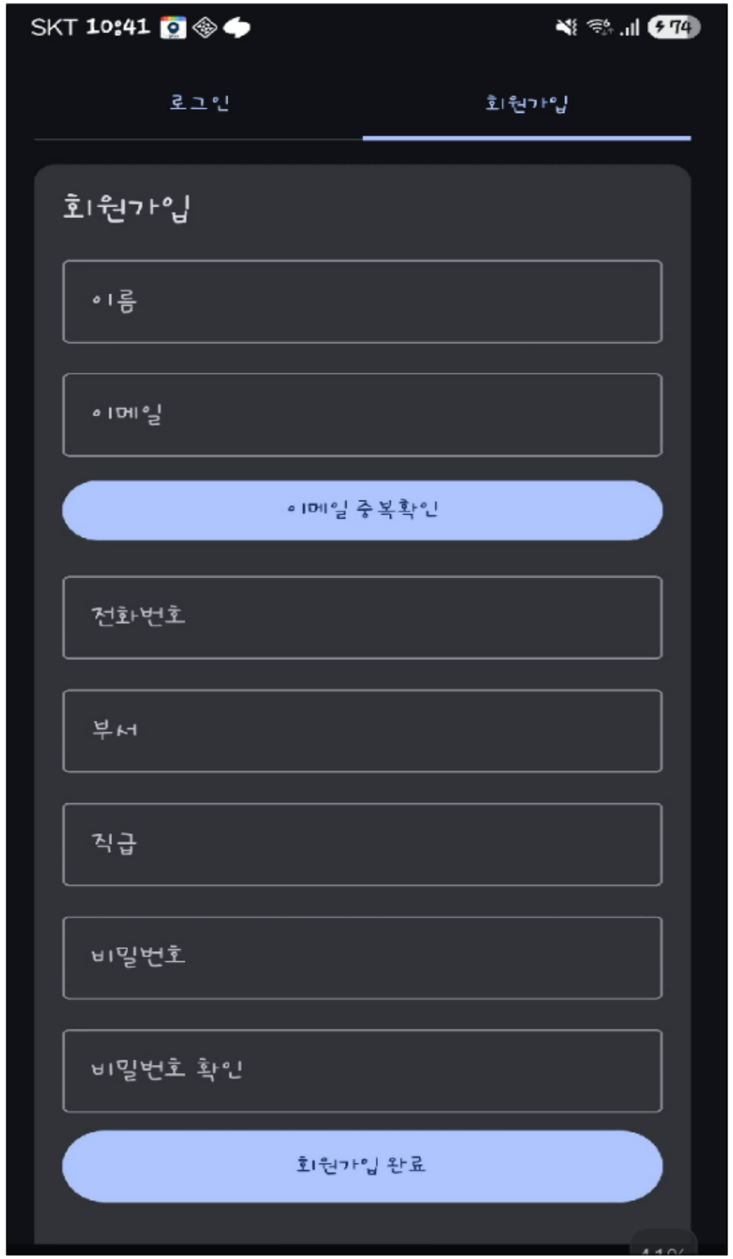
if check_password_hash(employee["password_hash"], user_pw):
    token = create_token(employee)
    return jsonify({"success": True, "token": token})
```

- 사용자가 입력한 이메일/사번을 기준으로 직원 정보를 DB에서 조회

- 저장된 비밀번호 해시값과 입력 비밀번호를 비교하여 로그인 여부를 검증

- 인증 성공 시 JWT 토큰을 발급하여 이후 직원 사원증 앱 접근에 사용

03. 프로젝트 결과 - 직원 사원증 - 회원가입




```python
password_hash = generate_password_hash(password)

cur.execute("""
    INSERT INTO employees
    (name, email, phone, department, position, role, password_hash, is_active, created_at)
    VALUES (%s, %s, %s, %s, %s, %s, %s, 1, NOW())
""", (name, email, phone, department, position, role, password_hash))

conn.commit()

return jsonify({
    "success": True,
    "message": "회원가입 완료"
})
```

password_hash
scrypt:32768:8:fgegregfrdhtehjtehfdgfdsr
scrypt:32768:8:1$V4QhikptgApR0ZJA$2d527f4...
scrypt:32768:8:1$djfFHbLu0E0sLO6y$f36aaf0...
scrypt:32768:8:1$SaBvB9CqTBiaOR24$fccd04f...
scrypt:32768:8:1$fnDC2cTcDQCizY5e$3416d2...

- 직원 정보를 DB에 저장하고 비밀번호는 해시 처리
- 직원 정보 기반으로 QR 데이터를 생성
- 가입 완료 후 회원가입 성공 메시지를 앱으로 반환

03. 프로젝트 결과 - 직원 사원증 - 사원 정보




```
var qrData = employee.optString("qr_data")

if (qrData.isBlank() || qrData == "null") {
    qrData = JSONObject().apply {
        put("type", "employee_access")
        put("employee_id", employeeId)
        put("email", email)
        put("name", name)
    }.toString()
}

val qrBitmap = createQrBitmap(qrData, 700)
```

- 로그인 성공 후 사용자 정보를 기반으로 사원증 화면을 구성

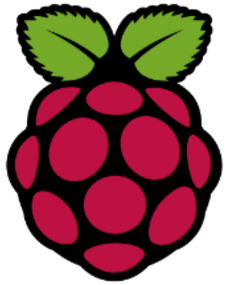
- 사원증에는 이름, 사번, 이메일, 부서, 직급, 권한 정보를 표시

- QR 코드는 이후 출퇴근 인증 및 직원 확인 기능에 활용

03. 프로젝트 결과 - 통제실_흐름



1.센서 감지

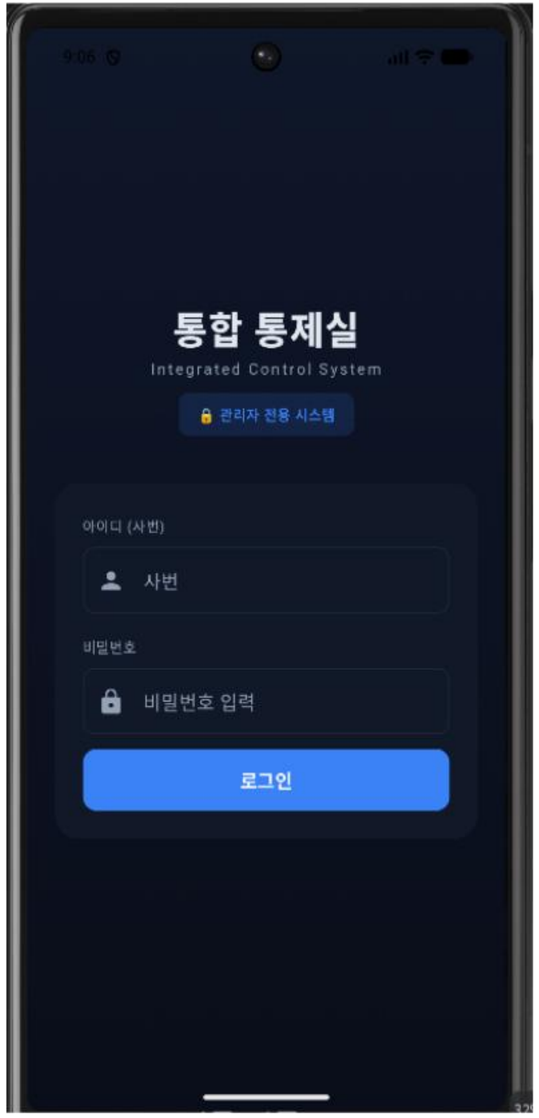




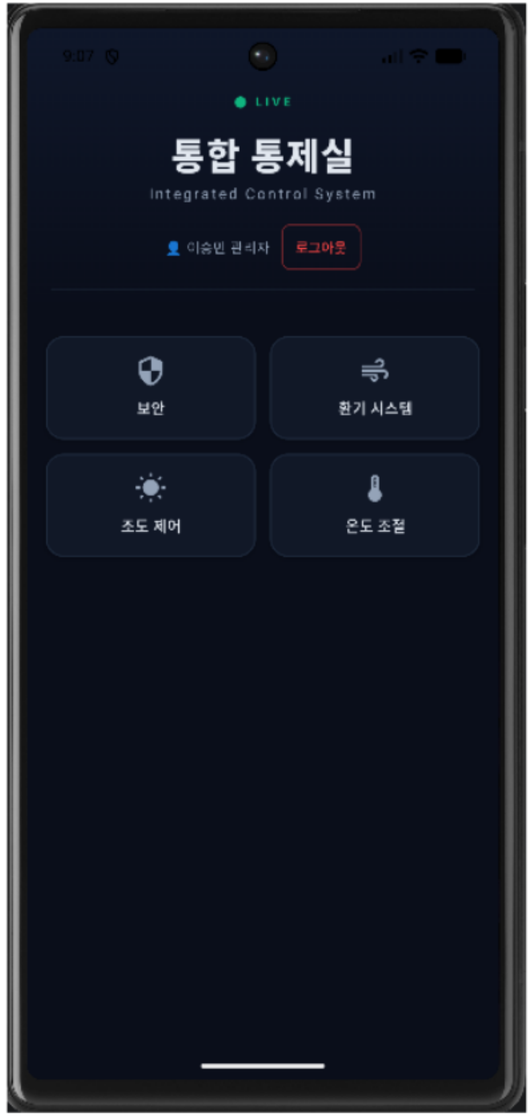
2. 라즈베리파이 / 아두이노 처리



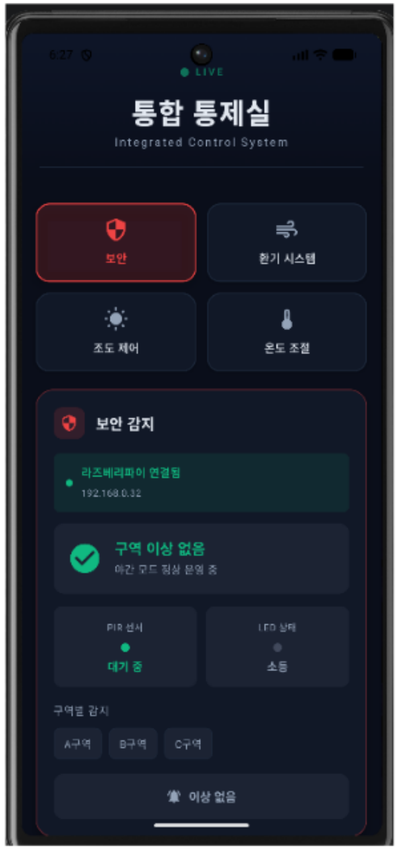
3. 앱에 실시간 전송
-> 앱에서 원격 제어

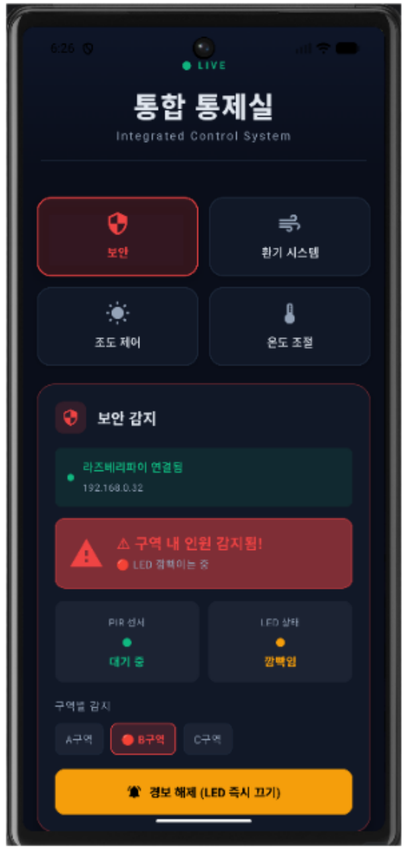


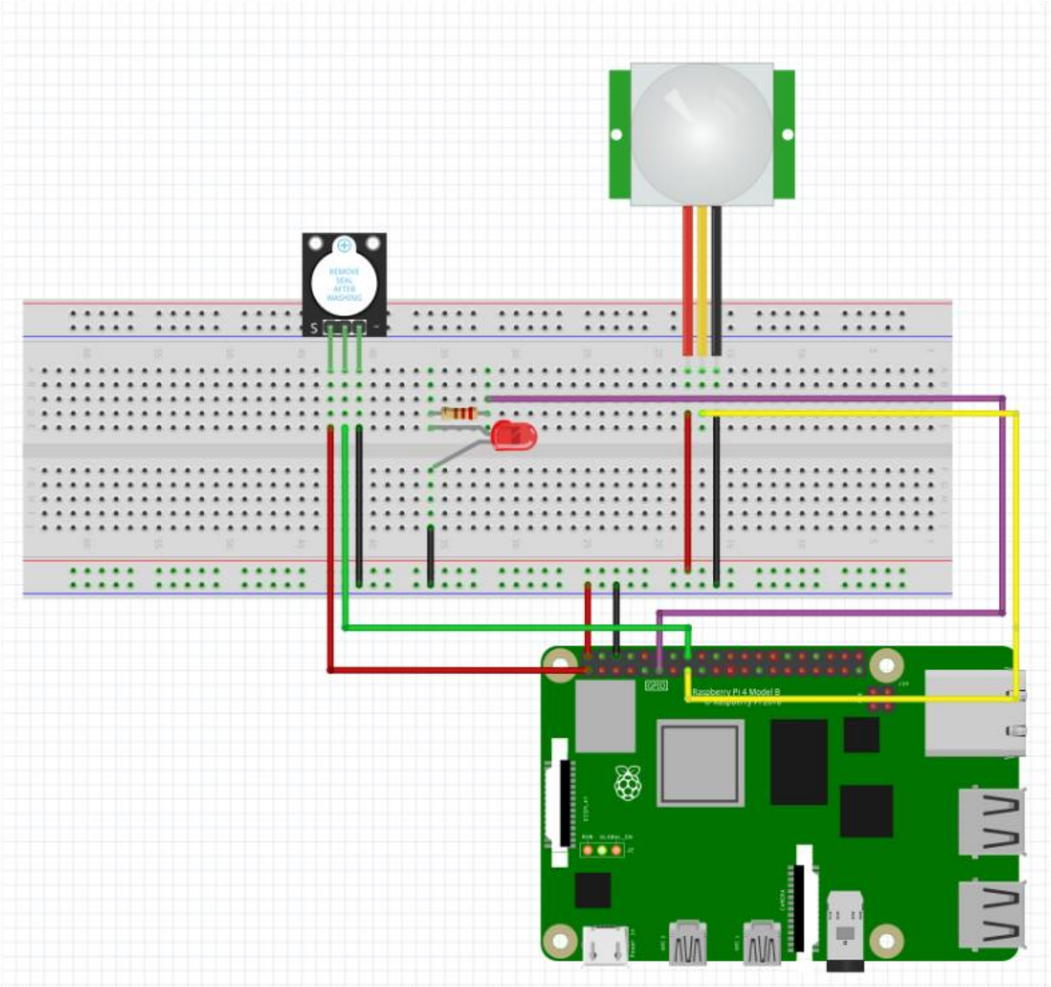




03. 프로젝트 결과 - 통제실_보안







Android Studio

```
// 1초마다 PIR 상태 확인
val res = httpGet("/status")
alarmActive = res.optBoolean("alarm_active")

// 경보 해제 버튼
httpPost("/alarm/acknowledge")
```

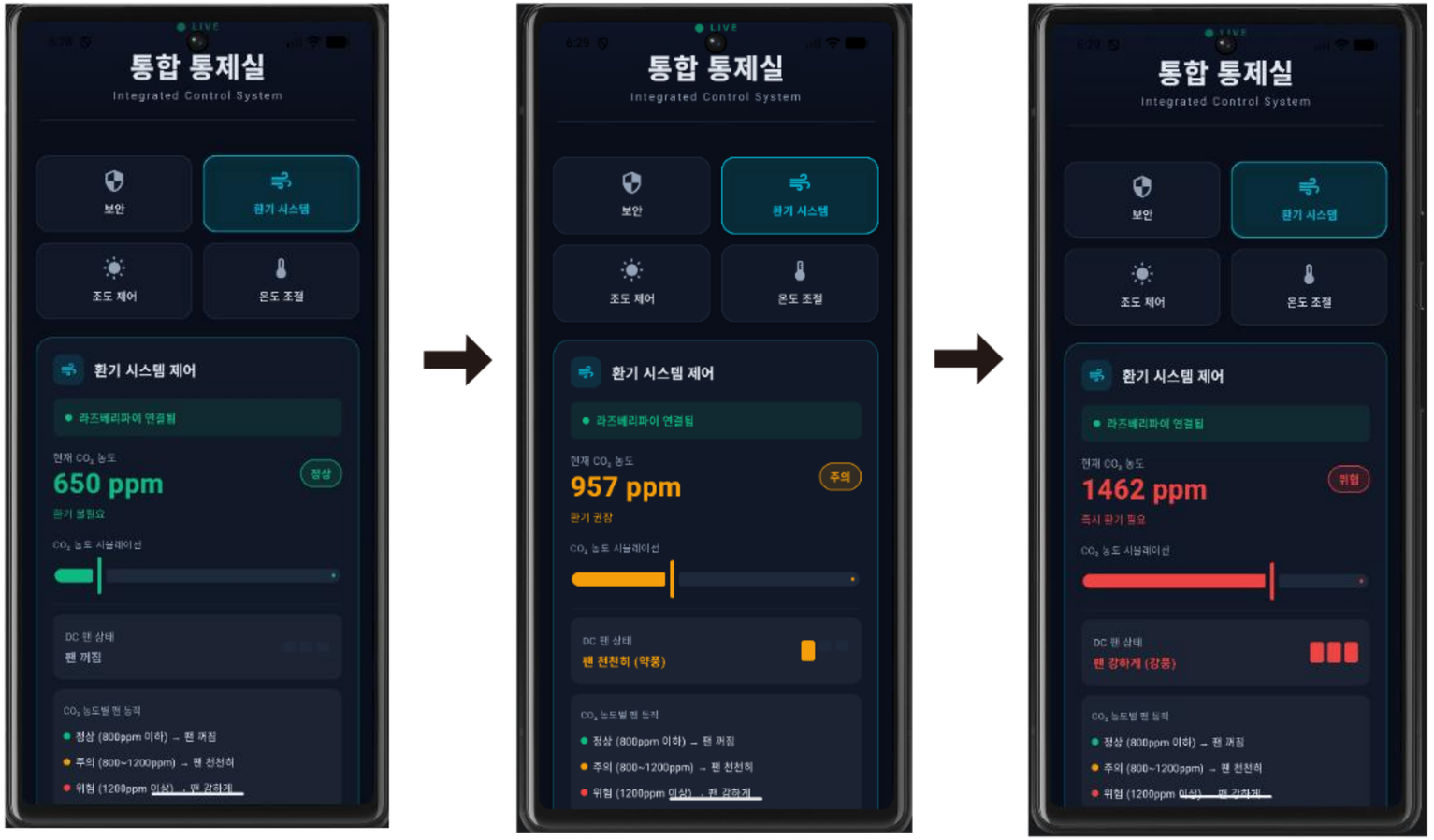
RPI_Visual Code

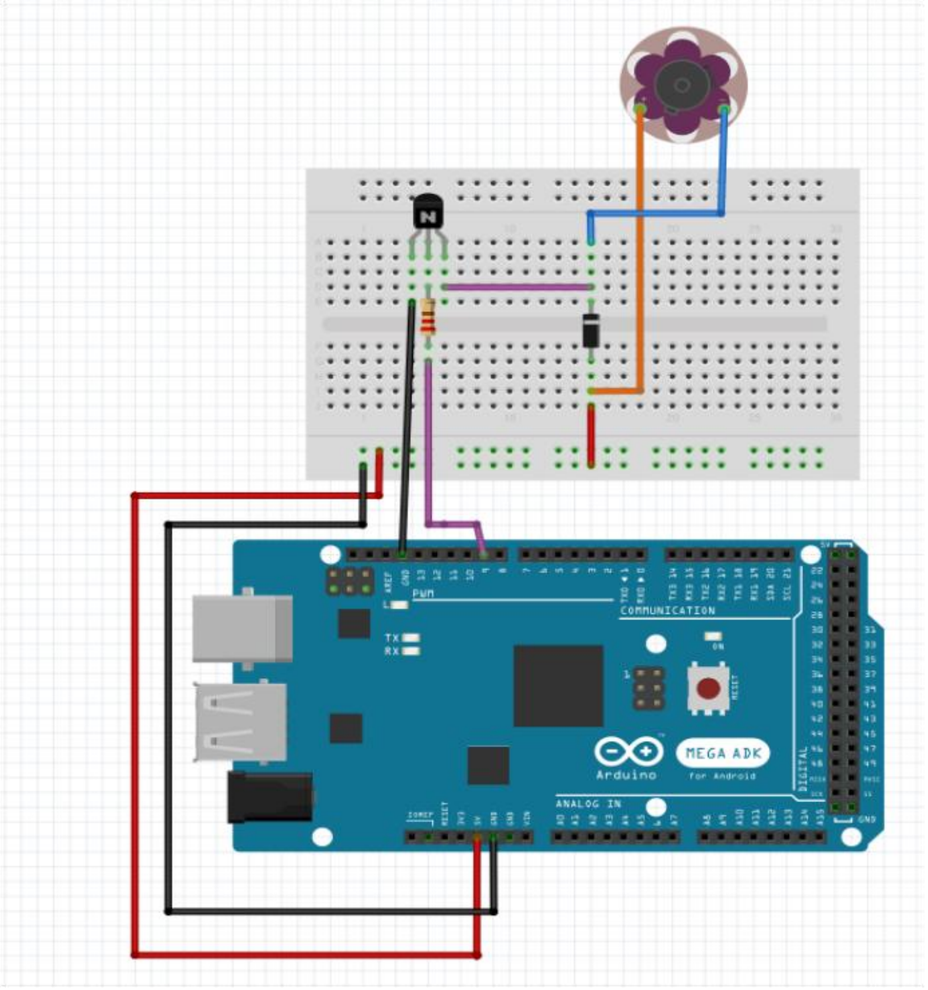
```
# PIR 감지 → LED + 부저 작동
def pir_loop():
    pir = GPIO.input(PIR_PIN) == GPIO.HIGH
    if pir and not prev_pir and not state["led_blinking"]:
        state["alarm_active"] = True
        state["led_blinking"] = True
        threading.Thread(target=blink_led, daemon=True).start()

# 경보 해제 → LED + 부저 즉시 꺼짐
@app.route("/alarm/acknowledge", methods=["POST"])
def acknowledge():
    GPIO.output(LED_PIN,    GPIO.LOW)   # LED 꺼짐
    GPIO.output(BUZZER_PIN, GPIO.HIGH)  # 부저 꺼짐
```

1. PIR 감지
2. 라파 LED + 부저 작동
3. 앱 경보 표시
4. 경보 해체 버튼
5. 즉시 꺼짐

03. 프로젝트 결과 - 통제실_환기 시스템





RPI_Visual Code

```
# CO2 값에 따라 팬 자동 제어
def update_fan_by_co2(co2):
    if co2 < 800:
        arduino.write(b"off\n")    # 팬 꺼짐
    elif co2 < 1200:
        arduino.write(b"slow\n")   # 약풍
    else:
        arduino.write(b"fast\n")   # 강풍
```

Arduino IDE

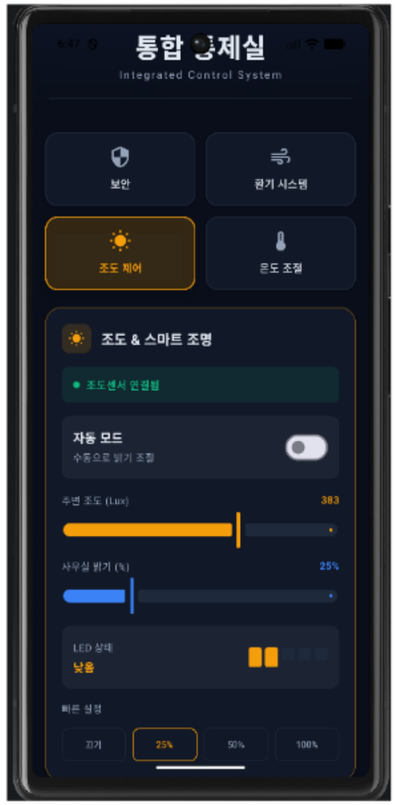
```
// 라파 명령 받아서 팬 속도 제어
String cmd = Serial.readStringUntil('\n');
if (cmd == "off")  analogWrite(fanPin, 0);    --> 정지
if (cmd == "slow") analogWrite(fanPin, 180);  --> 약풍
if (cmd == "fast") analogWrite(fanPin, 255);  -->강풍
```

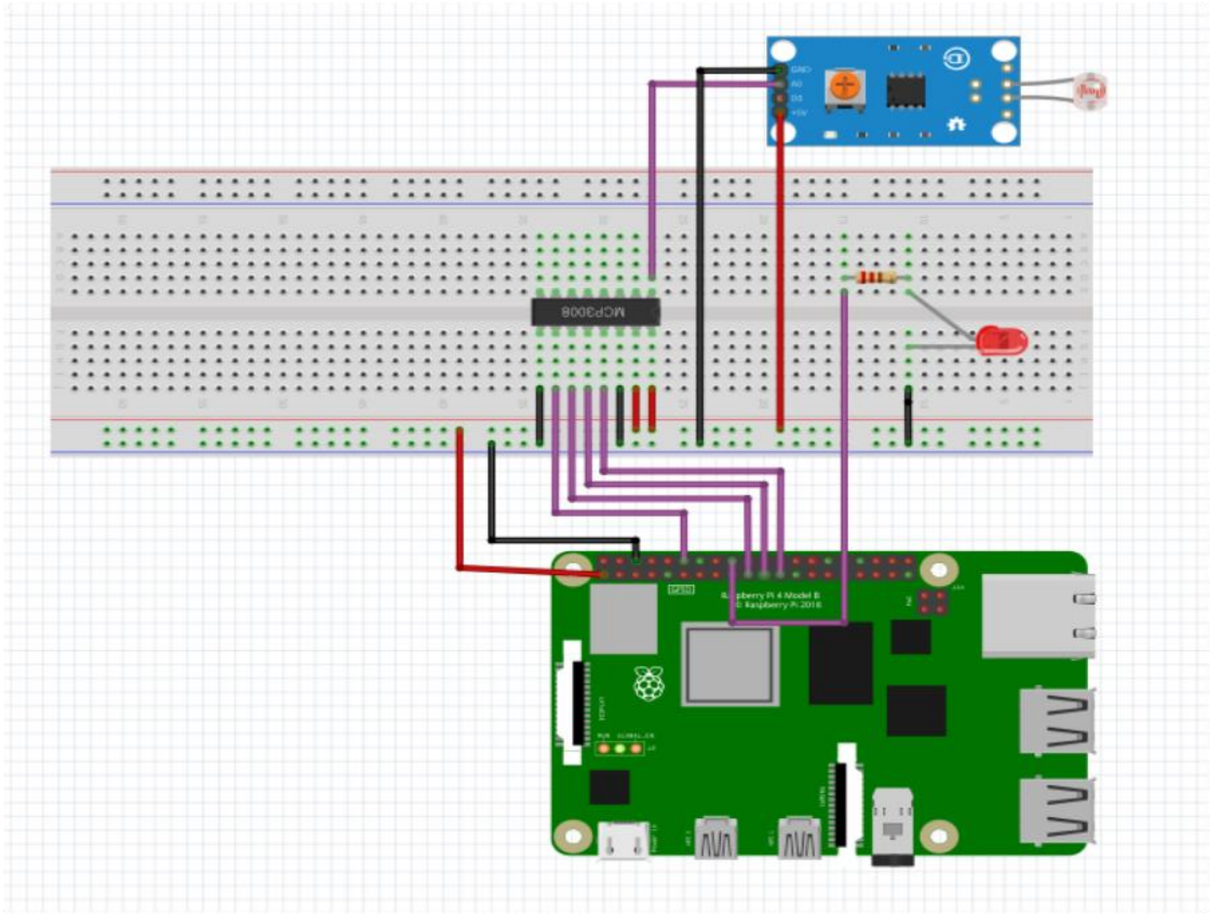
Android Studio

```
// 슬라이더 조절 시 라파로 전송
httpPost("/ventilation/co2",
    JSONObject().put("co2", co2Level.toInt()))
```

03. 프로젝트 결과 - 통제실_조도제어







RPI_Visual Code

```
# 조도센서 값 읽기 (MCP3008)
def read_light():
    adc = spi.xfer2([1, (8 + 0) << 4, 0])
    return ((adc[1] & 3) << 8) + adc[2]

# 자동 모드 - 어두울수록 LED 밝게
if state["auto_mode"]:
    brightness = int((1023 - val) / 1023 * 100)
    pwm.ChangeDutyCycle(brightness)
```

Android Studio

```
// 자동 모드 ON/OFF
httpPost2("/light/auto",
    JSONObject().put("auto_mode", newVal))

// 수동 밝기 조절
httpPost2("/light/brightness",
    JSONObject().put("brightness", ledBrightness.toInt()))
```

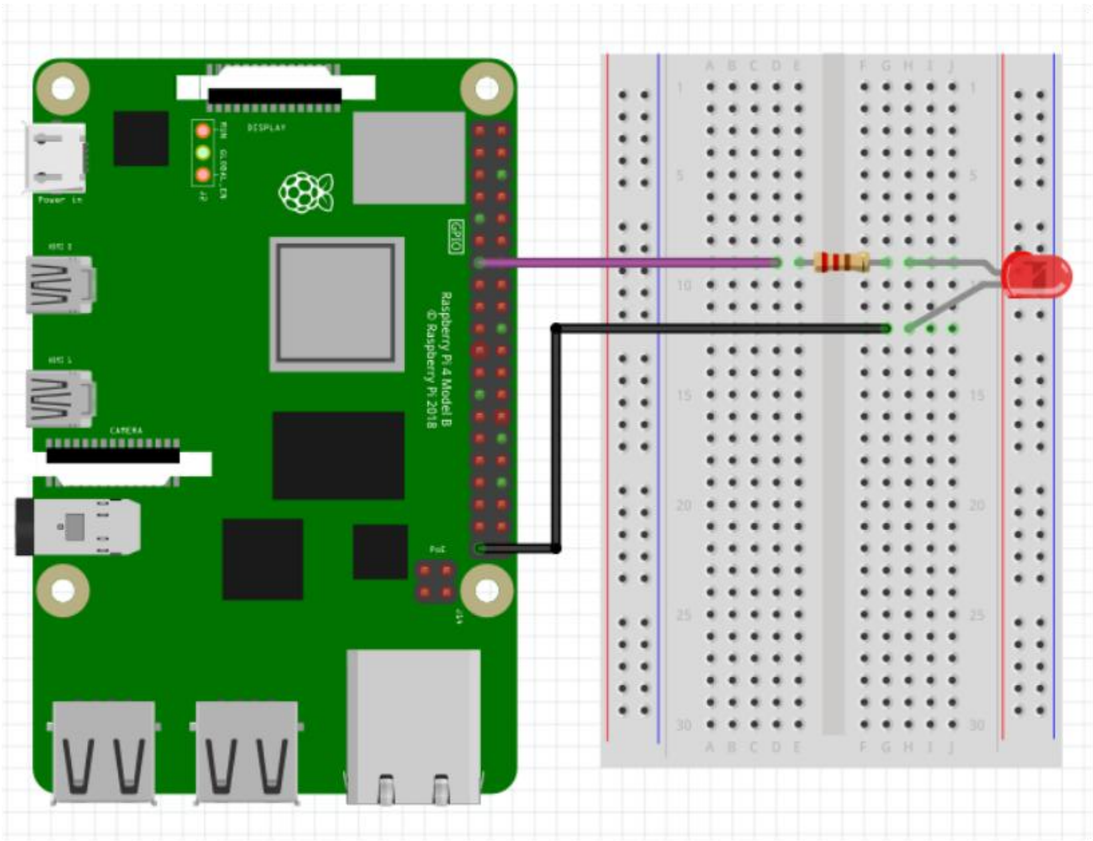
조도센서 밝기 측정 ➡ 자동 모드 ON ➡ LED 자동 조절

자동모드 OFF ➡ 앱 슬라이더 및 버튼 수정 조절

03. 프로젝트 결과 - 통제실_온도조절







RPI_Visual Code

```
# 앱에서 모드 받아서 LED 제어
@app.route("/ac/mode", methods=["POST"])
def set_ac_mode():
    mode = request.json.get("mode")
    if mode == "off":
        GPIO.output(AC_LED_PIN, GPIO.LOW)  # LED 꺼짐
    else:
        GPIO.output(AC_LED_PIN, GPIO.HIGH) # LED 켜짐
```

Android Studio

```
// 시스템 시작/정지 버튼
val modeEn = when (mode) {
    "냉방" -> "cool"
    "난방" -> "heat"
    else   -> "fan"
}
httpPost("/ac/mode",
    JSONObject().put("mode", modeEn))
```

앱에서 모드 + 시스템 시작 버튼

➡ 라파로 명령 전송

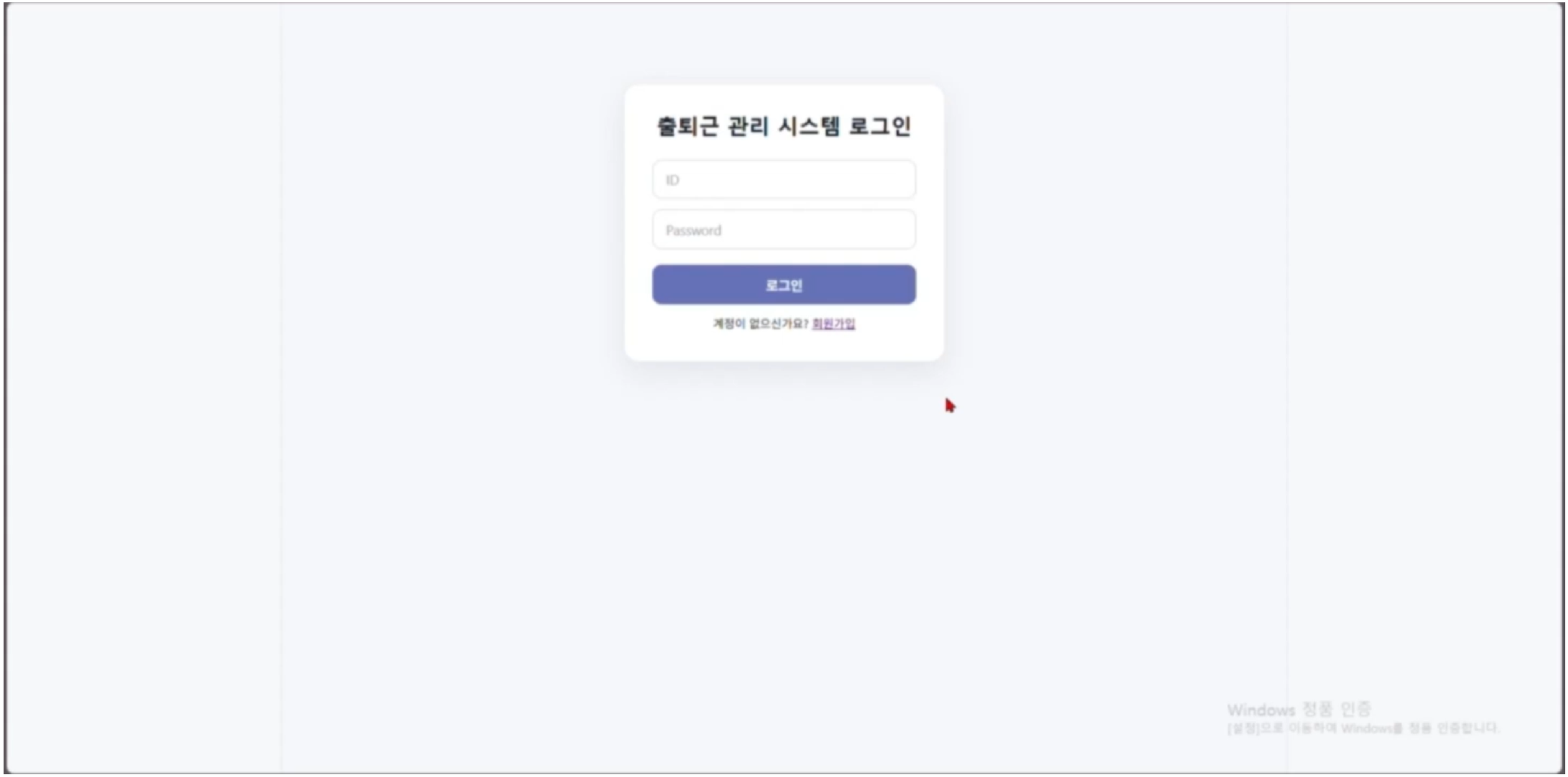
➡ 에어컨 LED켜짐(작동 중 표시)

➡ 정지 버튼--> LED꺼짐

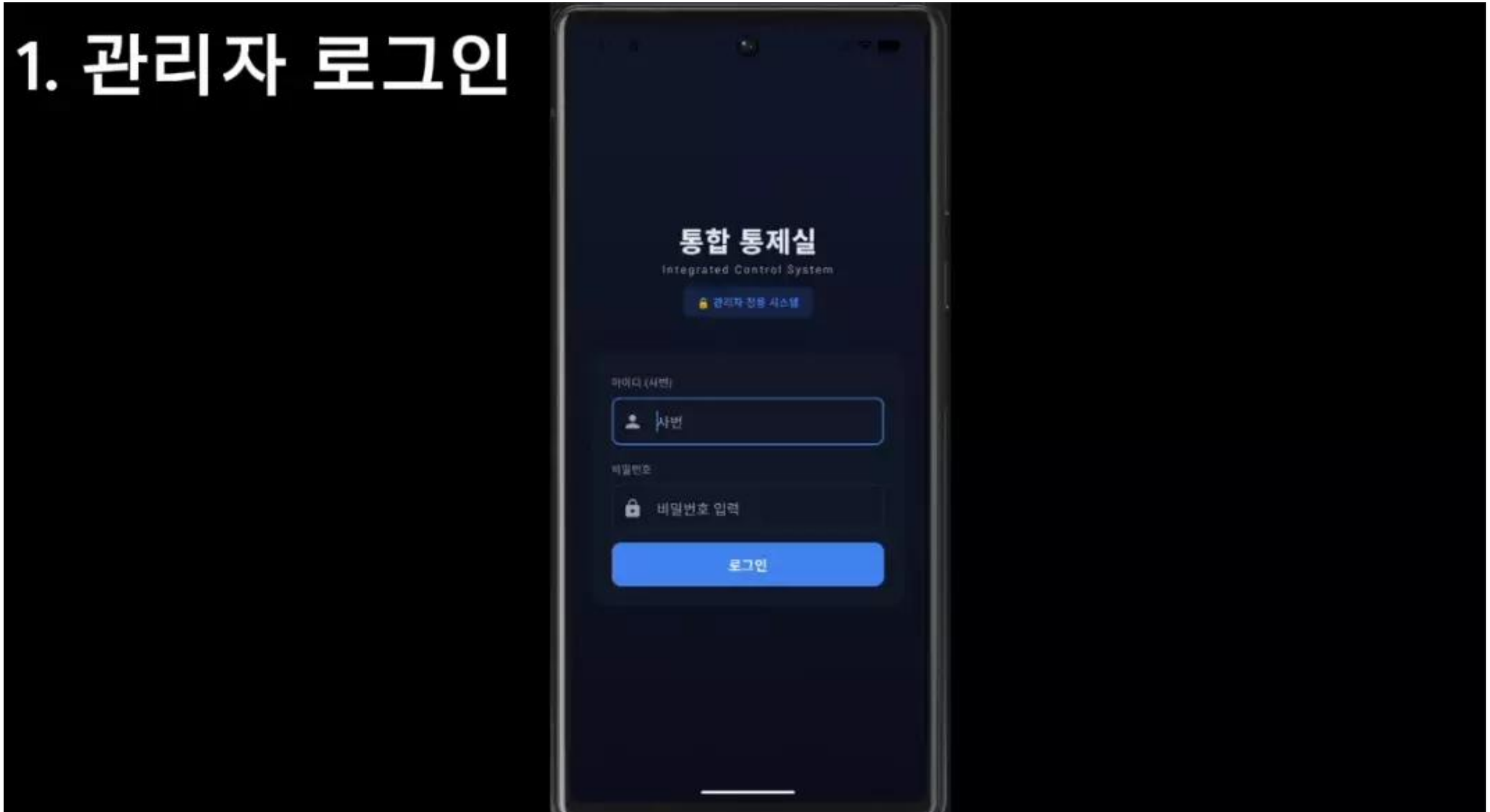
chaper 06.

시연영상

06. 시연영상 - WEB



06. 시연영상 - 관리자 App



chaper 07.

프로젝트 후기

07. 프로젝트 후기

김수하

프로젝트를 진행하며 단순히 화면을 만드는 것을 넘어, 기능들이 서로 연결되어 하나의 시스템으로 동작하는 과정을 경험할 수 있었습니다.
팀원들과 역할을 나누어 개발하고 결과물을 통합하는 과정에서 협업과 소통의 중요성을 느꼈습니다.

이승민

이번 팀 프로젝트를 통해 역할 분담과 소통의 중요성을 느꼈고, 각 기능이 하나의 시스템으로 연결되는 과정을 배울 수 있었습니다.
오류를 함께 해결하며 협업 능력과 문제 해결 능력을 키울 수 있었던 좋은 경험이었습니다.

강은비

새로운 프레임워크와 언어를 사용하다보니
알아가는 과정에서
구현 하지 못한 기능들에 대해
아쉬움부터듭니다.
하지만 그만큼 제가 무엇을
더 공부해야하는지 깨달을 수 있는
의미 있는 기간이였습니다.

김민준

처음으로 웹 ui를 만들어 보고,
또 단순한 하드코딩이 아닌
DB를 기반으로 브라우저에
정보 표시 및 기능 처리를 수행하였는데,
처음 만들어보는 유형의 프로그램을 통해
새로운 언어와 툴을 공부해볼 수 있어서
뜻깊은 시간이었습니다.

정지수

라즈베리파이와 아두이노를 다루면서
하드웨어와 소프트웨어가 연결되는 과정을
직접 경험할 수 있었고 센서와 안드로이드 앱을
실시간으로 통신하는 시스템을 구현하는 과정이 재미있었습니다.

chaper 08.

Q & A

감사합니다